REAL TIME ACCIDENT MONITORING AND RISK PREDICTION

A Project Report

Submitted by:

BEJJUPURAMU SUSEELA (228T1A0516)

BOGGAVARAPU MANASA LAVANYA (228T1A0519)

MADDALI VENKATA RAGA NANDINI (228T1A0566)

BEESABATHUNI S N MANJUNATH (228T1A0514)

BOYAPATI VENKATA SURAJ (228T1A0522)

Under The Esteemed Guidance Of

Dr. G. KRISHNA KISHORE

Professor, Department of CSE

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

at

DHANEKULA INSTITUTE OF ENGINEERING AND TECHNOLOGY

GANGURU, A.P. (INDIA) - 521139

AFFILIATED TO JNTUK KAKINADA, ANDHRA PRADESH (INDIA)

(NAAC-A, Accredited by NBA (For Beach CSE | ECE))

APRIL 2026

DECLARATION

I hereby declare that the project entitled **"REAL TIME ACCIDENT MONITORING AND RISK PREDICTION"** submitted for the B. Tech. (CSE) degree is our original work and the project has not formed the basis for the award of any other degree, diploma, fellowship or any other similar titles.

BEJJUPURAMU SUSEELA	(228T1A0516)	-
BOGGAVARAPU MANASA LAVANYA	(228T1A0519)	-
MADDALI VENKATA RAGA NANDINI	(228T1A0566)	-
BEESABATHUNI S N MANJUNATH	(228T1A0514)	-
BOYAPATI VENKATA SURAJ	(228T1A0522)	-

Place: VIJAYAWADA

Date:

DHANEKULA INSTITUTE OF ENGINEERING & TECHNOLOGY

AUTONOMUS

(Approved by AICTE, New Delhi, Affiliated to JNTUK, Kakinada)

(NAAC-A, Accredited by NBA(For B. TECH CSE| EEE| MECH| ECE))

GANGURU:: VIJAYAWADA -521 139

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the project titled "**REAL TIME ACCIDENT MONITORING AND RISK PREDICTION**" is the bonafide work carried out by **Bejjupuramu Suseela (228T1A0516), Boggavarapu Manasa Lavanya (228T1A0519), Maddali Venkata Raga Nandini (228T1A0566), Beesabathuni S N Manjunath (228T1A0514), Boyapati Venkata Suraj (228T1A0522)** of B.Tech (CSE), Dhanekula Institute of Engineering and Technology, during the academic year 2025-2026, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

SIGNATURE OF THE GUIDE
Dr. G. Krishna Kishore
Professor, Department of CSE

SIGNATURE OF THE HOD
Dr. K. Sowmya
HOD & Professor

SIGNATURE OF THE EXTERNAL EXAMINER

VISION-MISSION-PEOs

Institute Vision	Pioneering Professional Education through Quality
Institute Mission	Providing Quality Education through state-of-art infrastructure, laboratories and committed staff. Moulding Students as proficient, competent, and socially responsible engineering personnel with ingenious intellect. Involving faculty members and students in research and development works for betterment of society.
Department Vision	To empower students of Computer Science and Engineering Department to be technologically adept, innovative, global citizens possessing human values.
Department Mission	Encourage students to become self-motivated and problem-solving individuals. Prepare students for professional career with academic excellence and leadership skills. Empower the rural youth with computer education. Create Centre's of excellence in Computer Science and Engineering.
Program Educational Objectives (PEOs)	Graduates of B.Tech (Computer Science & Engineering) will be able to **PEO1**: Excel in Professional career by demonstrating the capabilities of solving real time problems through Computer-based system, Machine learning and allied software applications. **PEO2**: Able to pursue higher education and research. **PEO3**: Communicate effectively, recognize, and incorporate appropriate tools and technologies in the chosen profession. **PEO4**: Adapt to technological advancements by continuous learning, team collaboration and decision making.

PROGRAM OUTCOMES(POs)

PO1. Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals, and an engineering specialization to develop the solution of complex engineering problems. (WK1 to WK4)

PO2. Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3. Design / Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society, and environment as required. (WK5)

PO4. Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis and interpretation of data to provide valid conclusions. (WK8)

PO5. Engineering Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6. The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7)

PO7. Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national and international laws. (WK9)

PO8. Individual and Collaborative Team-work: Function effectively as an individual, and as a member or leader in diverse teams/multi-disciplinary teams.

PO9. Communication: Communicate effectively and inclusively within engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, considering cultural, language, and learning differences.

PO10. Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11. Life-Long Learning: Recognize the need for and have the preparation and ability for i). independent and life-long learning ii). Adaptability to new and emerging technologies and iii). Critical thinking in the broadest context of technological change. (WK8)

Knowledge and Attitude Profile (WK)

WK1: A systematic, theory-based understanding of the natural sciences applicable to the discipline and awareness of relevant social sciences.

WK2: Conceptually-based mathematics, numerical analysis, data analysis, statistics and formal aspects of computer and information science to support detailed analysis and modelling applicable to the discipline.

WK3: A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.

WK4: Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.

WK5: Knowledge, including efficient resource use, environmental impacts, whole-life cost, re-use of resources, net zero carbon, and similar concepts, that supports engineering design and operations in a practice area.

WK6: Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.

WK7: Knowledge of the role of engineering in society and identified issues in engineering practice in the discipline, such as the professional responsibility of an engineer to public safety and sustainable development

WK8: Engagement with selected knowledge in the current research literature of the discipline, awareness of the power of critical thinking and creative approaches to evaluate emerging issues.

WK9: Ethics, inclusive behavior and conduct. Knowledge of professional ethics, responsibilities, and norms of engineering practice. Awareness of the need for diversity by reason of ethnicity, gender, age, physical ability etc. with mutual understanding and respect, and of inclusive attitudes

PROGRAM SPECIFIC OUTCOMES (PSO' s)

PSO1: Have expertise in algorithms, networking, web applications and software engineering for efficient design of computer-based systems of varying complexity.

PSO2: Qualify in national international level competitive examinations for successful higher studies and employment.

PROJECT MAPPINGS

Batch No:	A4
Project Title	Real Time Accident Monitoring And Risk Prediction
Project Domain	Artificial Intelligence & Deep Learning with IOT
Type of the Project	Application
Guide Name	Dr.G.Krishna Kishore
Student Roll No	**Student Name**
228T1A0516	Bejjupuramu Suseela
228T1A0519	Boggavarapu Manasa Lavanya
228T1A0566	Maddali Venkata Raga Nandini
228T1A0514	Beesabathuni S N Manjunath
228T1A0522	Boyapati Venkata Suraj

COURSE OUTCOMES: At the end of the Course/Subject, the students will be able to

CO.No	Course Outcomes(COs)	POs	PSOs	Blooms Taxonomy & Level
R20C501.1	Identify the real world problem with a set of requirements to design a solution.	1,2	1,2	Apply- L3
R20C501.2	Implement, Test and Validate the solution against the requirements for a given problem.	3,5	1,2	Apply-L3
R20C501.3	Lead a team as a responsible member in developing software solutions for real world problems and societal issues with ethics.	3,6,7,8	1,2	Evaluate-L5
R20C501.4	Participate in discussions to bring technical and behavioral ideas for good solutions.	8,9,10	1,2	Understand -L2
R20C501.5	Express ideas with good communication skills during presentations.	8,9	1,2	Evaluate – L5
R20C501.6	Learn new technologies to contribute in the software industry for optimal solutions.	11	1,2	Create - L6

Course Outcomes vs PO`s Mapping:

Courses Out Comes	P01	P02	P03	P04	P05	P06	P07	P08	P09	P10	P011
R20C501.1	3	3	-	-	-	-	-	-	-	-	-
R20C501.2	-	-	3	-	3	-	-	-	-	-	-
R20C501.3	-	-	3	-	-	3	3	3	-	-	-
R20C501.4	-	-	-	-	-	-	-	3	3	3	-
R20C501.5	-	-	-	-	-	-	-	3	3	-	-
R20C501.6	-	-	-	-	-	-	-	-	-	-	3
Total	3	3	6	-	-	3	3	9	6	3	3
Average	3	3	3	-	3	3	3	3	3	3	3

Justification of Mapping of Course Outcomes with Program Outcomes:

R20C501.1: Course outcome maps with PO 1, PO 2 as the project involves identifying real-world road accident scenarios and analyzing traffic safety problems. Engineering knowledge and problem-analysis skills are applied to define system requirements for a real-time accident monitoring and risk prediction system using AI and IoT technologies.

R20C501.2: Course outcome maps with PO3 and PO5 as the proposed solution is designed, implemented, tested, and validated using modern engineering tools such as YOLOv8, computer vision libraries, and deep learning frameworks. The system is evaluated for accurate accident detection, risk prediction, and real-time alert generation**.**

R20C501.3: Course outcome maps with PO3, PO6, PO7, and PO8. The project is developed collaboratively by team members while considering ethical AI usage, data integrity, and public safety. Societal and environmental impacts of road accidents are evaluated, ensuring the solution contributes positively to society and promotes responsible engineering practices.

R20C501.4: Course outcome maps with PO8, PO9, and PO10. Active participation in technical discussions related to model selection, system architecture, and IoT integration helped improve decision-making. Behavioral and collaborative inputs supported effective teamwork and project management.

R20C501.5: Course outcome maps with PO8 and PO9. Project concepts, system workflow, and experimental results are communicated effectively through presentations, documentation, and demonstrations, enabling clear understanding of AI-based accident monitoring among evaluators and stakeholders.

R20C501.6: Course outcome maps with PO11 as the project requires learning and applying emerging technologies such as deep learning, YOLOv8, computer vision, and IoT systems. Continuous learning and adaptability are demonstrated to develop innovative and optimized solutions in the smart transportation domain.

Course Outcomes vs PSOs Mapping:

Courses Out Comes	PSO1	PSO2
R20C501.1	3	3
R20C501.2	3	3
R20C501.3	3	3
R20C501.4	3	3
R20C501.5	3	3
R20C501.6	3	3
Total	18	18
Average	3	3

Justification of Mapping of Course Outcomes with Program Specific Outcomes:

All course outcomes aligns with PSO1 and PSO2 by applying expertise in algorithms, networking, web applications, and software engineering to design a complex, computer-based real-time system. Advanced algorithms and deep learning models such as YOLOv8 and CNN-LSTM are used for accident detection and risk prediction, while networking concepts enable real-time communication between the AI system and IoT devices for alert generation. The project follows systematic software engineering practices to integrate video processing, AI models, and alert mechanisms into an efficient solution. Additionally, hands-on exposure to Artificial Intelligence, computer vision, IoT integration, and real-time system development strengthens analytical and problem-solving skills, thereby enhancing preparedness for national and international competitive examinations, higher studies, and employment.

Mapping Level	Mapping Description
1	Low Level Mapping with PO & PSO
2	Moderate Mapping with PO & PSO
3	High Level Mapping with PO & PSO

228T1A0516 – BEJJUPURAMU SUSEELA
228T1A0519 - BOGGAVARAPU MANASA LAVANYA
228T1A0566 – MADDALI VENKATA RAGA NANDINI **Project Guide**
228T1A0514 – BEESABATHUNI S N MANJUNATH
228T1A0522 - BOYAPATI VENKATA SURAJ

ACKNOWLEDGEMENT

Behind every achievement lies an unfathomable sea of gratitude to those who activated it, without whom it would ever have come into existence. To them we lay the words of gratitude imprinted with us.

We would like to thank our respected Principal, **Dr. Ravi Kadiyala**, and **Dr. K. Sowmya**, Head of the Department, Computer Science and Engineering, for their support throughout our major project.

It is our sincere obligation to thank our guide**, Mr. Dr. G. Krishna Kishore**, Department of Computer Science and Engineering, for his timely valuable guidance and suggestions for this major project.

We express our gratitude to **Dr. M. Sridevi Sameera**, Associate Professor and Project Coordinator, for giving us the opportunity and support to make this project successful.

We also extend our thanks to all the faculty members of the Computer Science & Engineering department for their valuable contributions in this project.

We would like to extend our warm appreciation to all our friends for sharing their knowledge and valuable contributions in this project.

Finally, we express our deep sense of gratitude to our parents for their continuous support throughout our academic career and their encouragement in the completion of this project successfully.

Bejjupuramu Suseela (228T1A0516),
Boggavarapu Manasa Lavanya (228T1A0519),
Maddali Venkata Raga Nandini (228T1A0566),
Beesabathuni S N Manjunath (228T1A0514),
Boyapati Venkata Suraj (228T1A0522).

ABSTRACT

Road accidents are one of the major causes of injuries and loss of life, often due to delayed detection and emergency response. Traditional accident monitoring systems mainly detect accidents only after they occur, which may delay medical assistance. To address this issue, this project proposes a Real-Time Accident Monitoring and Risk Prediction System that combines IoT-based sensing, weather-aware risk analysis, and LSTM-based sequence learning for road safety support.

The system uses an ESP32 microcontroller as the core unit, along with an MPU6050 accelerometer, GPS module, crash switch, panic button, LCD display, buzzer, and motor for vehicle representation. The ESP32 continuously monitors vehicle motion, sudden impact, crash switch status, panic input, and GPS location. The collected sensor data is sent to a Python-based monitoring system, where live weather parameters such as rain, precipitation, wind speed, and visibility are considered for risk estimation. A stacked LSTM model is used to analyze sequential sensor and environmental data using a sliding window approach and classify accident risk into LOW, MEDIUM, and HIGH levels.

When a risk or accident condition is detected, the system provides immediate local alerts through the LCD and buzzer. It also updates a live web dashboard and sends emergency notifications through Telegram, including severity level and a Google Maps location link based on GPS coordinates. This hybrid approach connects hardware-based real-time accident detection with software-based risk prediction and alert communication, helping to reduce response delay and improve road safety awareness.

Keywords: Accident Detection, IoT, ESP32, MPU6050, LSTM, Weather-Aware Risk Prediction, GPS, Telegram Alert, Real-Time Monitoring, Smart Transportation.

LIST OF FIGURES

Figure No.	Name of the Figure	Page No.
Fig 3.1	Block Diagram	33
Fig 3.2	Use Case Diagram	34
Fig 3.3	Class Diagram	35
Fig 3.4	Activity Diagram	36
Fig 3.5	State Machine Diagram	37
Fig 3.6	Sequence Diagram	38
Fig 3.7	Component Diagram	39
Fig 3.8	Deployment Diagram	40
Fig 3.9	Flow Diagram	41
Fig 4.1	Dashboard Output -1	59
Fig 4.2	Dashboard Output-2	59
Fig 4.3	Dashboard Output-3	60
Fig 4.4	Dashboard Output-4	60
Fig 4.5	Hardware Output -1	61
Fig 4.6	Hardware Output -2	61
Fig 4.7	Alert-1	62
Fig 4.8	Alert-2	62

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Pin Configuration	30
Table 2	Test Case 1	55
Table 3	Test Case 2	55
Table 4	Test Case 3	56
Table 5	Test Case 4	56
Table 6	Test Case 5	57

CONTENTS

TITLE PAGE	i
DECLARATION OF THE STUDENT	ii
CERTIFICATE OF THE STUDENT	iii
VISION-MISSION-PEO'S	iv
PROJECT MAPPINGS	vii
ACKNOWLEDGEMENT	xi
ABSTRACT	xii
LIST OF FIGURES	xiii
LIST OF TABLES	xiv
1.INTRODUCTION	**1**
1.1 Problem Statement	5
1.2 Project Overview	6
1.2.1 Origin of the problem	9
1.2.2 Scope	10
1.2.3 Basic Concepts	11
1.3 Hardware Specification	13
1.4 Software Specification	13
2.LITERATURE SURVEY	**15**
2.1 Existing System	19
2.2 Proposed System	20
2.3 Feasibility Study	22
2.3.1 Technical Feasibility	22

2.3.2 Economical Feasibility 23

2.3.3 Social Feasibility 24

3.SYSTEM ANALYSIS & DESIGN 25

3.1 Requirement Specification 26

3.1.1 Functional Requirements 28

3.1.2 Non- Functional Requirements 29

3.1.3 Pin Configuration 30

3.2 Flowcharts/UML Diagram 31

3.2.1 Block Diagram 33

3.2.2 Use Case Diagram 33

3.2.3 Class Diagram 35

3.2.4 Activity Diagram 36

3.2.5 State Machine Diagram 37

3.2.6 Sequence Diagram 38

3.2.7 Component Diagram 39

3.2.8 Deployment Diagram 40

3.2.9 Flow Diagram 41

3.3 Design and Test Steps / Criteria 42

3.3.1 Implementation and Setup Details 43

3.4 Algorithms and Pseudo Code 48

3.4.1 Sensor Data Acquisition and Telemetry Algorithm 48

3.4.2 Risk Prediction and Alert Generation Algorithm 49

3.5 Testing Process 50

3.5.1 Types of Testing 50

3.5.1.1 Functional Testing 50

3.5.1.2 Performance Testing 51

3.5.1.3 Security Testing 51

3.5.1.4 Usability Testing 51

3.5.1.5 Regression Testing 52

3.5.1.6 System Testing 52

3.5.1.6.1 White Box Testing 52

3.5.1.6.2 Black Box Testing 53

3.5.2 Levels of Testing 53

3.5.2.1 Unit Testing 53

3.5.2.2 Integration Testing 54

3.5.2.3 Acceptance Testing 54

3.5.3 Test Cases 55

4.RESULTS 58

4.1 Output Screens 59

5.CONCLUSIONS/ RECOMMENDATIONS 63

6.FUTURE ENHANCEMENTS 65

7. REFERENCES 67

8.PUBLISHED PAPER 70

INTRODUCTION

INTRODUCTION

Road accidents have become one of the most serious global public safety challenges, causing a significant number of injuries, fatalities, and economic losses every year. With the rapid increase in the number of vehicles, traffic congestion, and unsafe driving practices, the chances of road accidents are continuously rising. In many cases, the severity of an accident is not only due to the collision itself but also because of the delay in accident detection, lack of early risk prediction, and slow emergency response. Conventional traffic monitoring systems and manual surveillance methods are mostly reactive in nature, meaning they identify accidents only after they occur. Such systems do not provide prior warnings about risky situations and therefore fail to prevent accidents or reduce their impact effectively.

In recent years, advancements in Artificial Intelligence (AI), Computer Vision, Deep Learning, and Internet of Things (IoT) have opened new possibilities for developing intelligent transportation and safety systems. AI-based computer vision models can analyze live video streams from CCTV cameras or dashcams to identify vehicles, detect abnormal movement, estimate speed, and understand traffic patterns in real time. At the same time, IoT devices can be used to provide immediate alerts, location tracking, and communication with emergency services. By combining these technologies, it is possible to build a system that not only detects accidents after they occur but also predicts accident risk before the event happens, allowing timely preventive action and faster medical assistance.

The proposed project, “Real-Time Accident Monitoring and Risk Prediction using AI & IoT,” is developed to address this critical problem by integrating intelligent video analysis with real-time alert mechanisms. The system uses YOLOv8 (You Only Look Once Version 8) for real-time object detection and CNN-LSTM (Convolutional Neural Network with Long Short-Term Memory) models for analyzing temporal traffic behavior such as vehicle motion, speed changes, and interaction patterns. These models help in identifying dangerous situations and estimating the probability of an accident in advance. If the system detects a high-risk scenario or confirms an accident, it activates an IoT-based alert module consisting of ESP32, GPS, and buzzer components. The buzzer can immediately warn the driver or nearby personnel, while the GPS module can provide location details and enable SOS messages or notifications to emergency contacts, hospitals, or rescue services.

This project aims to bridge the gap between software-based intelligent prediction and hardware-based real-time response. Unlike traditional systems that focus only on post-accident detection, the proposed system emphasizes both accident monitoring and risk prediction, making it more proactive and effective. It can be used in smart transportation systems, highways, industrial fleet monitoring, and future intelligent vehicle safety applications. By reducing the time taken to detect dangerous situations and initiate alerts, the system has the potential to

improve road safety, minimize accident severity, reduce emergency response delays, and ultimately save human lives.

Characteristics of the Proposed System

1. Accuracy and Risk Detection Precision

Accuracy is one of the most important characteristics of the proposed system. The project uses a **stacked LSTM deep learning model** for accident risk prediction by analyzing sequential sensor and environmental data. Real-time inputs from the **MPU6050 sensor, crash switch, panic button, GPS module, and weather parameters** improve prediction reliability. Threshold checks and severity mapping help reduce false alerts and improve the precision of accident and risk detection.

2. Real-Time Monitoring and Processing

The system is designed to operate in **real time** by continuously collecting and processing sensor telemetry from the **ESP32-based IoT module**. Data from motion sensing, crash detection, panic input, and GPS tracking is sent instantly to the Python-based monitoring system. The system performs live risk evaluation and updates the dashboard without delay, enabling quick detection of dangerous conditions.

3. AI-Based Risk Prediction

The project incorporates **Artificial Intelligence and Machine Learning** using a **stacked LSTM model** to predict accident risk before an actual crash occurs. By using a **sliding window approach**, the model analyzes sequential sensor patterns and environmental conditions to classify the risk level into **LOW, MEDIUM, and HIGH**. This predictive capability makes the system more advanced than traditional accident detection systems.

4. IoT-Based Automation

The proposed system uses an **ESP32 microcontroller** as the central IoT device for collecting data from multiple hardware components. It automates accident monitoring by reading sensor values, detecting crash conditions, receiving panic input, and transmitting telemetry to the software layer. This reduces dependency on manual observation and improves automation in emergency response systems.

5. Weather-Aware Risk Estimation

To improve accident prediction, the system integrates **live weather data** such as **rain, precipitation, wind speed, and visibility**. These environmental parameters are combined with sensor telemetry to estimate accident risk more accurately. This weather-aware approach enhances the intelligence of the system by considering external road safety conditions that may increase accident probability.

6. Hybrid Hardware and Software Integration

The system combines **hardware-based accident detection** with **software-based risk prediction and alert communication**. The ESP32 and connected sensors handle physical event monitoring, while the Python application performs data processing, LSTM-based prediction, dashboard visualization, and Telegram alerting. This hybrid design improves both detection capability and practical usability.

7. Real-Time Sensor Monitoring

The project continuously monitors multiple inputs such as **accelerometer motion values, crash switch status, panic button state, and GPS location**. These inputs help in identifying both actual accidents and risky situations. Continuous monitoring ensures that the system can respond quickly to abnormal motion, collision-like conditions, or manual emergency triggers.

8. Local Alert Mechanism

The system provides **immediate local alerts** through the **LCD display and buzzer** whenever a warning or accident condition is detected. The LCD displays the current status and GPS details, while the buzzer provides an audible alert for quick attention. This ensures that alerts are available even if remote communication is temporarily unavailable.

9. Automated Telegram Notification System

The project uses **Telegram-based alerting** to send real-time emergency notifications automatically. When a high-risk or accident condition is detected, the system sends an alert containing the **severity level, accident status, and Google Maps location link**. This enables fast communication with emergency contacts or monitoring personnel without manual intervention.

10. Live Dashboard Visualization

A **web-based live dashboard** is used to display the current state of the system in real time. It shows **system status, risk score, weather details, accelerometer readings, GPS coordinates, and alert information**. This improves monitoring efficiency and provides a user-friendly interface for demonstrations and practical observation.

11. GPS-Based Location Tracking

The system integrates a **GPS module** to provide real-time **latitude and longitude** information. During warning or accident conditions, the GPS data is used to generate a **Google Maps location link**, which is included in the dashboard and Telegram alerts. This feature helps responders quickly identify the accident location.

12. Severity Level Classification

The project includes a **severity mapping mechanism** to classify accident conditions based on sensor intensity and trigger events. The system determines whether the event is **normal, warning, or accident**, and displays the corresponding severity level. This helps in prioritizing emergency response based on the seriousness of the situation.

13. Controlled Failure Handling

The system is designed to handle controlled failure conditions such as **GPS unavailability, Telegram API failure, or internet connectivity loss**. Even if remote alerting fails, local alerting through the **LCD and buzzer** continues to function. This improves the reliability and robustness of the system during real-world operation.

14. Performance Optimization

The project is optimized for continuous monitoring and fast response. Sensor acquisition, serial communication, risk evaluation, dashboard updates, and alert generation are designed to work efficiently together. The system ensures smooth performance during normal monitoring as well as during accident or warning conditions.

15. Scalability

The system is designed using a **modular architecture**, making it easy to extend and improve in future versions. Additional sensors, improved AI models, cloud logging, mobile applications, and emergency network integration can be added without redesigning the complete system. This makes the project scalable for larger real-world deployments.

16. User-Friendly Operation

The system operates automatically with **minimal user intervention**. It provides clear outputs through the **LCD display, dashboard interface, buzzer alerts, and Telegram notifications**. The administrator or evaluator can easily understand the system status, risk level, and alert information without requiring deep technical knowledge.

17. Security and Configuration Management

Sensitive values such as **Telegram bot tokens, chat IDs, and API keys** are securely stored using **environment variables (.env file)**. This prevents hardcoding of confidential information in the source code. Proper configuration handling improves system security and makes the project easier to manage.

18. Efficiency and Time Saving

The automated nature of the system significantly reduces the delay involved in accident reporting. Risk detection, accident identification, location tracking, and alert generation are performed instantly. This saves time, reduces dependency on manual reporting, and improves the possibility of faster emergency response.

19. Reliability

The proposed system is designed for **stable and continuous operation**. It can continuously monitor sensor data, process risk logic, and generate alerts without frequent interruption. Error handling and fallback mechanisms ensure that the system remains functional even during partial communication failures.

20. Cost-Effectiveness

The project is developed using **low-cost and easily available components** such as **ESP32, MPU6050, GPS module, crash switch, LCD, buzzer, and open-source Python technologies**. This makes it an economical and practical solution for student-level implementation as well as future low-cost deployment in real-world environments.

1.1 PROBLEM STATEMENT

Road accidents are a major cause of injuries and fatalities, and in many cases, they remain undetected for several minutes or even hours, especially in remote or less-monitored areas. This delay in accident detection significantly affects emergency response time and reduces the chances of timely medical assistance. Traditional road safety systems lack real-time monitoring, accident prediction, and instant alert generation, making them ineffective in preventing accidents or minimizing their impact.

In addition, most existing solutions focus either on software-based accident analysis or hardware-based accident detection, but do not provide a unified approach that combines both intelligent prediction and real-time response. There is therefore a strong need for an integrated system that can continuously monitor traffic conditions, predict accident risk in advance, detect accidents instantly when they occur, and provide immediate alerts with precise location information. Hence, this project proposes a Real-Time Accident Monitoring and Risk Prediction System using AI and IoT to improve road safety, reduce emergency response delays, and save lives.

1.2 PROJECT OVERVIEW

The proposed project, “Real-Time Accident Monitoring and Risk Prediction using AI & IoT,” is designed to develop an intelligent and integrated road safety system that combines IoT-based real-time monitoring with AI-based risk prediction. The system uses an ESP32-based IoT module to collect critical parameters such as motion data, crash detection signals, panic alerts, and GPS location information from the vehicle or monitoring unit. These sensor readings are transmitted to a Python-based monitoring application through serial communication, where the incoming telemetry is continuously processed and analyzed. In addition to sensor data, the system also fetches live weather information to improve the accuracy of accident risk analysis by considering environmental conditions such as rain, fog, or poor visibility.

The Python monitoring application acts as the central control unit of the system. It computes the risk level and severity level of a possible accident by analyzing the combined sensor and environmental data, updates a real-time dashboard interface, and generates Telegram alerts whenever a high-risk or crash event is detected. As part of the research and intelligent prediction component, the project uses a stacked LSTM (Long Short-Term Memory) model to analyze sequential patterns in sensor values and environmental features for risk classification. This deep learning model helps in identifying dangerous conditions in advance and supports early warning before an accident occurs. Overall, the system provides a hybrid solution that integrates hardware-based detection, software-based monitoring, machine learning-based prediction, and real-time alert communication, making it a practical and effective approach for improving road safety and emergency response.

OBJECTIVES

- To monitor vehicle motion and accident-related inputs in real time.
- To classify accident risk into LOW, MEDIUM, and HIGH severity levels.
- To include weather conditions such as rain, wind, and visibility in risk analysis.
- To generate local alerts through LCD and buzzer.
- To send remote Telegram alerts with severity and Google Maps location.
- To provide a live dashboard for demo and monitoring purposes.

Methodology:

1. Data Collection and Preprocessing

- The project uses a hybrid data collection approach that combines dataset-based model development with real-time IoT sensing. For the research and model-training component, the TAD (Traffic Accident Detection) dataset is used as the reference dataset to study accident-related traffic patterns and sequential event behavior. This dataset provides benchmark inspiration for identifying accident-prone situations and supports the development of the LSTM-based risk prediction model. For the live prototype, real-time sensor data is collected using an ESP32-based IoT module. The ESP32 acquires motion data from the MPU6050 sensor, crash status from the limit switch, panic input from the push button, and location data from the GPS module. On the software side, the Python program reads serial telemetry and fetches live weather parameters such as rain, precipitation, wind speed, and visibility using the Open-Meteo API. The collected data is cleaned, normalized, and arranged into fixed-length sequential windows of 30 timesteps for temporal analysis.

2. Feature Engineering

- Feature engineering is used to transform raw motion and environmental readings into meaningful input features for risk prediction. Accelerometer values along the X, Y, and Z axes are used to derive motion intensity, acceleration magnitude, and impact-related changes. Additional event indicators such as crash switch status and panic input are also considered in the live system. Environmental features including rain, precipitation, wind speed, and visibility are added to capture external conditions that may increase accident probability. The resulting feature vector combines both vehicle behavior and environmental context, which helps the model distinguish between LOW, MEDIUM, and HIGH-risk situations more effectively.

3. Model Selection and Training

- A stacked LSTM (Long Short-Term Memory) model is selected because accident risk depends on temporal patterns rather than isolated sensor readings. The TAD dataset serves as the benchmark reference for accident-related sequential analysis, while the LSTM model is designed to capture time-dependent trends in vehicle motion and environmental features. The input data is organized using a sliding window of 30 timesteps and passed through two LSTM layers, followed by dropout and dense layers for classification. The output layer uses SoftMax activation to predict one of three classes: LOW, MEDIUM, or HIGH risk. The model is trained using a standard train-validation-test split and optimized with Adam and categorical cross-entropy loss.

4. System Deployment and Implementation

- The implemented system consists of both hardware and software components. The ESP32 continuously gathers motion, crash, panic, and GPS data and sends it to the Python monitoring application through serial communication. The Python system performs risk estimation, integrates live weather information, updates the web dashboard, and triggers alerts. When a warning or accident condition is detected, the system provides local alerts using the LCD and buzzer, and remote alerts using Telegram with severity level and Google Maps location. This combination of dataset-inspired model development and real-time IoT deployment makes the system suitable for both research evaluation and practical demonstration.

5. Testing and Validation

- Testing is carried out at multiple levels to verify both the machine learning and real-time monitoring components.

  Types of Testing Conducted

  a. Unit Testing

  - ESP32 sensor acquisition validated (MPU6050, crash switch, panic button, GPS).
  - Accident trigger logic verified from motion and crash inputs.
  - Severity mapping validated: crash switch -> LOW, sudden impact -> MEDIUM, vigorous shake -> HIGH.
  - Weather risk score rule evaluation verified.

  b. Integration Testing

  - Serial JSON telemetry from ESP32 to Python verified.
  - State command handshake tested (CMD: CLEAR, CMD: PREALERT, CMD: ACCIDENT, ACK).
  - Telegram API integration tested (send Message).
  - Dashboard live JSON update (live_state.json) verified.
  - Google Maps link generation from latitude/longitude verified.

  c. Functional Testing

  - Crash switch trigger generated accident alert successfully.
  - Panic button manual trigger activated emergency alert successfully.
  - MPU6050 vigorous shake produced high-severity accident status.
  - LCD, buzzer, motor-stop, and dashboard outputs were verified during alert conditions.

  d. Performance Testing

- Real-time telemetry processing verified under continuous sensor updates.
- Alert response time observed to be suitable for demo-scale monitoring.
- Training and validation accuracy of the stacked LSTM model were monitored during model development.

e**. Reliability / Failure Testing**

- GPS unavailable condition handled without application crash.
- Internet / Telegram failure handled while local alerting continued.
- System remained stable during repeated test runs and manual reset cycles.

6.Expected Outcomes

- Development of a real-time accident monitoring system using ESP32, MPU6050, GPS, crash switch, and panic button for detecting abnormal vehicle conditions.
- Integration of weather-aware risk analysis and LSTM-based sequence prediction to classify accident risk into LOW, MEDIUM, and HIGH levels.
- Generation of immediate alerts through LCD, buzzer, live dashboard, and Telegram with severity level and Google Maps location for faster response.

Origin of the Problem

Road accidents have become one of the major causes of injuries, vehicle damage, and loss of human life in modern transportation systems. In many cases, the major problem is not only the accident itself, but the delay in detecting the accident and informing emergency services. Traditional accident reporting mostly depends on manual communication by drivers, passengers, or nearby people, which can consume valuable time during critical situations. This delay can reduce the chances of immediate medical support and increase the severity of consequences.

Another important issue is that many existing accident monitoring systems focus mainly on detecting accidents only after they happen. Such systems may identify a crash event, but they do not provide any early warning or pre-alert mechanism when a vehicle is entering a risky condition. In addition, some systems rely only on camera-based monitoring, which may fail under poor lighting, fog, rain, or low visibility conditions. Similarly, threshold-only sensor systems may detect sudden impacts but often fail to capture sequential motion patterns that indicate rising accident risk before an actual crash occurs.

These limitations created the need for a more advanced and intelligent system that can perform both real-time accident detection and accident risk prediction. The origin of this project lies in the need to combine IoT-based sensing, weather-aware analysis, and AI-based sequence learning into a single hybrid solution. By using ESP32-

based hardware, MPU6050 motion sensing, GPS tracking, weather parameters, and an LSTM-based prediction model, the proposed system aims to reduce emergency response delay, improve early warning capability, and enhance overall road safety monitoring.

FUTURE SCOPE

The proposed project “Real-Time Accident Monitoring and Risk Prediction using AI & IoT” provides a strong foundation for intelligent road safety systems, and it can be further enhanced in several ways in the future. One important future improvement is the direct integration with official emergency networks such as 100/108 through authorized APIs, which would enable the system to automatically send accident alerts to police, ambulance, or emergency response centers without relying only on third-party messaging platforms. This would make the system more reliable and suitable for real-world deployment.

Another valuable enhancement is the development of a mobile application for drivers and emergency contacts, allowing users to monitor live vehicle status, accident risk level, alert notifications, and GPS location directly from their smartphones.

In future versions, the system can also support cloud logging and long-term accident-risk analysis, where sensor data, alerts, and historical risk records are stored in the cloud for continuous monitoring, analytics, and trend-based safety improvements. This would help in identifying accident-prone conditions and improve decision-making over time.

From the machine learning perspective, the system can be improved by collecting a larger real-world dataset to enhance the training of the LSTM model, thereby increasing prediction accuracy and making the risk classification more robust in practical driving conditions.

In addition, camera-based accident context fusion can be added in future versions, where visual information from cameras is combined with sensor telemetry and weather data to provide richer accident context and more intelligent detection.

Finally, the system can be deployed in real vehicles with improved enclosure design and power management, making the hardware more durable, reliable, and suitable for continuous field operation.

Overall, the future scope of this project lies in making the system more intelligent, scalable, practical, and ready for real-world smart transportation applications.

1.2.1 Basic Concepts

Programming Language & Core Libraries

- The project uses two main programming environments: Arduino C/C++ for the ESP32 firmware and Python for the monitoring, risk analysis, alerting, and dashboard integration. The Arduino program is responsible for real-time sensor acquisition, GPS reading, LCD updates, buzzer control, and serial telemetry transmission. Python is used to process the serial data, integrate weather information, calculate risk score, derive severity level, update the dashboard, and send Telegram alerts.
- The core Python libraries used in the project include:
- **pyserial:** used for serial communication between ESP32 and the Python monitoring system.
- **requests**: used for API communication, including Open-Meteo weather data and Telegram alert requests.
- **python-dotenv:** used to load environment variables such as serial port, Telegram token, and chat IDs from the .env file.
- **Json**: used for telemetry parsing and dashboard state updates.
- **time and datetime**: used for timing, cooldown logic, and timestamp handling.
- **pathlib and os:** used for file handling and project path management.
- **HTML, CSS, and JavaScript:** used to build the live web dashboard that displays real-time system status and output.
- For the model and research part, the project conceptually uses a stacked LSTM model for sequence-based risk classification. This part is associated with deep learning model development and evaluation using standard machine learning and sequence-modeling concepts.

Advanced System Architecture & Operations

- The architecture of the project consists of two major layers: the hardware sensing layer and the software intelligence layer.
- In the hardware layer, the ESP32 interfaces with MPU6050, crash switch, panic button, GPS, LCD, buzzer, and motor. The ESP32 continuously reads motion and event data and converts it into structured serial telemetry. It also performs local control actions such as stopping the motor and activating the buzzer when an accident state is detected.
- In the software layer, the Python monitoring application receives telemetry from the ESP32 through serial communication. This application acts as the central processing unit for decision-making. It reads the incoming sensor data, fetches live weather conditions from Open-Meteo, derives risk score

and severity, updates the web dashboard, and sends Telegram alerts with severity and Google Maps location.

- For the research and model-analysis part, the architecture also includes a stacked LSTM-based prediction pipeline. The sequential sensor and environmental data are organized into sliding windows of 30 timesteps, passed to the LSTM model, and classified into LOW, MEDIUM, or HIGH risk. In the live demo, rule-based severity mapping and alert generation are integrated with this data-driven prediction concept, creating a practical hybrid system.
- Operationally, the system works in the following order: data acquisition, preprocessing, weather integration, risk estimation, severity mapping, local alerting, dashboard update, and remote Telegram alerting. This architecture ensures both real-time monitoring and intelligent decision support.

AI Paradigm Breakdown (The Hybrid Approach)

- The project follows a hybrid AI paradigm because it combines both data-driven prediction and rule-based real-time control.
- The data-driven component is the LSTM model. It is used for learning sequential patterns from accident-related data. Since accident risk evolves over time, LSTM is suitable because it can remember previous timesteps and identify temporal dependencies. The TAD dataset is used as the benchmark reference for understanding accident-related traffic patterns and inspiring the sequence-based risk modeling approach.
- The rule-based component is used in the live deployment system. The ESP32 and Python monitoring logic use predefined thresholds and event mappings to respond quickly during the demo. For example, crash switch activation maps to LOW severity, sudden impact maps to MEDIUM severity, and vigorous MPU6050 shaking maps to HIGH severity. Similarly, weather conditions contribute to a rule-based risk score for pre-alert generation.
- This hybrid design is important because a purely rule-based system may miss temporal patterns, while a purely model-based system may not be practical for fast embedded response in a student MVP. By combining both approaches, the project gains the advantages of real-time response, explainability, and sequence-based intelligence. Therefore, the system is both practically deployable for demonstration and academically meaningful for accident risk prediction research.

1.3 HARDWARE SPECIFICATION

- **Processor**: Intel i5 or higher (required for real-time YOLOv12 inference, face recognition processing, and multi-threaded execution).
- **RAM:** Minimum 8GB (to handle simultaneous frame buffering, face encoding cache, and Firebase upload operations efficiently)
- **Storage:** SSD 256GB minimum (for faster model loading, encoding cache storage, and captured evidence images)
- **GPU (Optional but Recommended):** NVIDIA GPU with CUDA support (improves YOLOv12 detection speed and reduces CPU load during inference)
- **Power Supply:** Reliable power backup (UPS/Inverter) to prevent interruption during continuous monitoring and detection.
- **ESP32:** Main controller and telemetry source
- **MPU6050:** Acceleration, motion, tilt, and impact sensing
- **16x2 LCD JHD162A:** Displays system status and alerts
- **NEO-6M GPS:** Provides latitude and longitude
- **Active Buzzer:** Audible warning and accident alert
- **Crash Switch / Limit Switch:** Simulates collision trigger.
- **Panic Button:** Manual emergency trigger
- **Small Motor:** Vehicle representation, stopped during accident state
- **Power Supply:** 5V-12V depending on module requirements.

1.4 SOFTWARE SPECIFICATION

- **Programming Language:** Python 3.10+
- **Embedded Firmware:** Arduino (ESP32)
- **Python Libraries:** pyserial (ESP32 to Python communication), requests (Weather and Telegram API calls), python-dotenv (Environment variable loading)
- **ESP32 Libraries:** Wire, TinyGPS++, Liquid Crystal
- **APIs/Services:** Open-Meteo Weather API, Telegram Bot API
- **Frontend:** HTML, CSS, JavaScript (real-time dashboard via JSON)
- **Development Environment:** Arduino IDE, VS Code / Command Prompt

- **Operating System:** Windows 10/11
- **Open-Meteo API:** Live weather data
- **Telegram Bot API:** Emergency alert communication
- **LSTM Model**: Sequence-based risk prediction

LITERATURE SURVEY

2.LITERATURE SURVEY

1. TAD: A LARGE-SCALE BENCHMARK FOR TRAFFIC ACCIDENTS DETECTION FROM VIDEO SURVEILLANCE

Published in: IEEE Access, 2025

Paper Proposed: This paper proposed TAD, a large-scale benchmark dataset for traffic accident detection using surveillance-camera video data. It includes multiple accident types and supports image classification, video classification, and object detection tasks.

Paper Summary: The authors highlighted the lack of large and diverse public datasets for accident detection, especially from surveillance cameras and highway scenarios. They created the TAD dataset to support computer vision research in intelligent transportation systems and benchmarked mainstream visual models on it. This paper is important because it provides a strong dataset reference for accident-related temporal and visual analysis.

Research Gap Identified: The paper mainly focuses on dataset creation and vision benchmarking. It does not provide a lightweight real-time IoT deployment, sensor-based accident monitoring, weather-aware risk estimation, or instant alerting through Telegram/dashboard like your project does.

Source: IEEE Access / ProjectBasepaper.pdf

2. DOTA: UNSUPERVISED DETECTION OF TRAFFIC ANOMALY IN DRIVING VIDEOS

Published in: IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI), 2023

Paper Proposed: This paper proposed an unsupervised anomaly detection framework for driving videos based on future object localization, and introduced the DoTA benchmark dataset.

Paper Summary: The method predicts future positions of traffic participants and uses inconsistencies in those predictions to identify anomalies. It is useful for video anomaly detection in dynamic driving scenes and contributes a large benchmark dataset for traffic anomaly analysis.

Research Gap Identified: The work is centered on dashcam video anomaly detection and does not include embedded sensing, real-time hardware triggering, weather-aware risk scoring, or practical alert generation for accident response.

Source: PubMed / TPAMI abstract

3. A VISION-BASED SYSTEM FOR TRAFFIC ANOMALY DETECTION USING DEEP LEARNING AND DECISION TREES

Published in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2021

Paper Proposed: The paper proposed a vision-based traffic anomaly detection system using YOLOv5 for object detection and a decision tree for final anomaly analysis.

Paper Summary: The authors combined deep learning and decision-tree logic to detect anomalies and estimate the start and end time of anomalous traffic events. Their work demonstrates good performance in traffic anomaly detection using camera feeds.

Research Gap Identified: This approach depends heavily on computer vision pipelines, which may be computationally expensive for low-cost deployments. It does not include ESP32-based sensing, GPS-triggered location alerts, or weather-integrated risk prediction, which are central to your project.

Source: CVPRW Open Access

4. LONG SHORT-TERM MEMORY NEURAL NETWORK FOR TRAFFIC SPEED PREDICTION USING REMOTE MICROWAVE SENSOR DATA

Published in: Transportation Research Part C: Emerging Technologies, 2015

Paper Proposed: This paper proposed an LSTM-based neural network for short-term traffic speed prediction using remote microwave traffic sensor data.

Paper Summary: The study showed that LSTM is highly effective for traffic-related time-series prediction because it can capture long-term temporal dependencies better than traditional methods such as ARIMA, SVM, and conventional RNN models. This work supports the use of LSTM for sequential traffic analysis.

Research Gap Identified: The paper focuses on traffic speed forecasting, not accident monitoring or risk-based emergency alerting. It also does not combine weather features, IoT hardware, or real-time accident notification mechanisms.

Source: ScienceDirect

5. HETERO-CONVLSTM: A DEEP LEARNING APPROACH TO TRAFFIC ACCIDENT PREDICTION ON HETEROGENEOUS SPATIO-TEMPORAL DATA

Published in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2018

Paper Proposed: This paper proposed Hetero-ConvLSTM, a deep learning model for traffic accident prediction using heterogeneous spatial and temporal urban data.

Paper Summary: The authors combined multiple contextual factors and spatio-temporal patterns to predict traffic accidents over a city-scale grid. The paper is significant because it shows how sequential deep learning can be applied to accident prediction rather than only accident detection.

Research Gap Identified: The approach is more suitable for large-scale urban prediction systems and requires more complex heterogeneous data. It is not designed as a low-cost embedded MVP, and it does not address onboard sensing, local alerts, Telegram notifications, or live dashboard integration.

Source: KDD listing / paper reference

6. THE RELATIONSHIP BETWEEN ROAD ACCIDENT SEVERITY AND RECORDED WEATHER

Published in: Journal of Safety Research, 1998

Paper Proposed: This paper studied the relationship between recorded weather conditions and road accident severity using accident records from England and Wales.

Paper Summary: The study examined how weather categories such as rain, fog, and high winds affect accident severity. It established that weather is an important contextual factor in road safety analysis and should not be ignored in accident-related studies.

Research Gap Identified: The paper is a statistical analysis study and does not implement a real-time system, sensor integration, machine learning model, or instant emergency alerting mechanism. Your project extends this idea by using weather as an active input to risk prediction.

Source: Journal of Safety Research / ScienceDirect

OVERALL RESEARCH GAP

From the above literature, it is clear that existing studies generally fall into one of the following categories:

- dataset and benchmark creation for accident detection,
- vision-based anomaly detection,
- traffic sequence prediction using LSTM,
- statistical weather-risk analysis, or
- large-scale spatio-temporal accident forecasting.

However, very few works combine low-cost IoT sensing, weather-aware risk prediction, LSTM-based sequential modeling, severity classification, GPS-based location sharing, and instant alert generation in a single practical MVP. This is the gap addressed by your project.

2.1 EXISTING SYSTEM

The existing accident monitoring systems are mostly reactive in nature and are designed to detect accidents only after they occur, rather than preventing them in advance.

One common approach is CCTV-based surveillance, where road cameras are used to monitor traffic and identify accidents visually. However, these systems can only detect accidents after the incident has already taken place, which results in late accident detection and consequently delayed medical assistance.

In many cases, accident reporting still depends on manual communication by eyewitnesses or drivers, which causes delays in emergency response and shows a high dependency on human intervention. Although camera-based systems can visually monitor traffic, they may become unreliable under poor lighting, fog, rain, or low-visibility conditions, reducing their effectiveness in real-world environments.

Some vehicles are equipped with basic crash sensors that work on predefined threshold values, but these hardware-based systems often function as standalone units without cloud connectivity, limiting their ability to share data or generate automated emergency alerts.

In addition, threshold-only hardware systems may fail to capture temporal risk patterns, such as gradual abnormal motion or changing driving behavior over time, which are important for early accident prediction. Similarly, GPS-based tracking systems provide only the location of the vehicle and do not include any mechanism to predict accidents or assess risk.

Another major limitation is that many existing systems do not consider environmental factors such as weather conditions, and therefore fail to perform weather-aware risk estimation, even though rain, poor visibility, and road conditions significantly affect accident probability.

Overall, existing systems suffer from a lack of integration between AI models and real-world vehicle sensors, provide limited automation in emergency alert generation, and do not support proactive accident prevention, making them less effective for real-time road safety management.

Disadvantages of Existing System

• Existing systems are mostly reactive and detect accidents only after they occur.
• They do not support early warning or proactive accident prevention.
• CCTV-based monitoring can only identify accidents after the incident happens.
• Camera-based systems may fail in poor lighting, fog, rain, or low-visibility conditions.
• Many systems depend on manual reporting by drivers or eyewitnesses, causing delay.
• High dependency on human intervention reduces system efficiency.
• Delayed accident detection leads to slower emergency response and medical assistance.
• Basic crash sensors work only on fixed threshold values, which may not be fully reliable.
• Threshold-only systems cannot capture temporal motion patterns or changing driving behavior.
• Many hardware systems work as standalone units without proper cloud or alert integration.
• Existing systems provide limited automation in emergency alert generation.
• GPS-based systems only provide location tracking and do not predict accident risk.
• Most existing systems do not consider weather conditions such as rain, wind, and visibility.
• Lack of weather-aware risk estimation reduces prediction accuracy in real road conditions.
• There is poor integration between AI models and real-world vehicle sensors.
• Overall, existing systems are less effective for real-time road safety management.

2.2 PROPOSED SYSTEM

The proposed system is an integrated AI and IoT-based real-time accident monitoring and risk prediction solution designed to improve road safety through both early warning and instant accident detection. Unlike traditional systems that focus only on post-accident response, the proposed system combines ESP32-based sensing, MPU6050 motion detection, GPS location tracking, weather-aware risk analysis, LSTM-based sequence learning, live dashboard visualization, and Telegram alerting to provide a complete and intelligent safety framework.

The Risk Prediction Module, which runs on a laptop-based Python application, continuously receives real-time sensor telemetry along with live weather parameters such as rain, wind speed, and visibility to estimate accident risk. Using a stacked LSTM model, the system analyzes sequential patterns in sensor and environmental data to identify risky conditions in advance and generates an early warning (PREALERT) whenever the risk level rises before an actual crash occurs. This enables proactive monitoring and helps reduce accident severity by warning users before the event happens.

The Accident Detection Module is implemented using an ESP32 microcontroller as the core IoT controller. The MPU6050 sensor detects sudden impact, abnormal motion, and vigorous shaking, while a crash/limit switch provides physical crash confirmation for more reliable accident detection.

A GPS module supplies real-time latitude and longitude coordinates, a panic button allows manual emergency triggering, and a buzzer with LCD display provides immediate local alerts and status information.

In addition, the Alert and Monitoring Module sends emergency notifications through Telegram, including accident severity and a Google Maps location link, while also displaying live system status on a web dashboard developed using HTML, CSS, JavaScript, and JSON.

By combining intelligent prediction with hardware-based crash detection, the proposed system provides real-time accident detection and prediction, enables faster emergency response through automated alerts, ensures accurate location tracking using GPS, reduces dependency on human reporting, and offers a hybrid, scalable, and cost-effective solution for modern road safety applications.

Advantages of Proposed System

- The proposed system provides both real-time accident detection and accident risk prediction.
- It supports early warning (PREALERT) before an actual accident occurs.
- The system is proactive, unlike traditional systems that are only reactive.
- It uses ESP32-based IoT sensing for continuous real-time monitoring.
- The MPU6050 sensor helps detect sudden impact, abnormal motion, and vigorous shaking.
- The crash/limit switch provides physical confirmation for more reliable accident detection.
- The system uses a stacked LSTM model for intelligent sequence-based risk prediction.
- It can identify risky conditions in advance by analyzing sequential sensor and weather data.
- The system performs weather-aware risk analysis using rain, wind speed, and visibility.
- It improves prediction accuracy by considering environmental conditions along with sensor inputs.
- The GPS module provides accurate real-time location tracking.
- It sends Google Maps location links for quick identification of accident location.
- The panic button allows manual emergency triggering during urgent situations.
- The LCD and buzzer provide immediate local alerts and status information.
- The system sends automated Telegram emergency notifications without manual intervention.
- It helps in reducing emergency response time through instant alert generation.
- It reduces dependency on human reporting by drivers or eyewitnesses.

• The live web dashboard provides real-time monitoring of system status and alerts.
• It offers a hybrid integration of AI and IoT, improving overall system intelligence.
• The system is scalable and can be extended for future real-world applications.
• It is cost-effective because it uses affordable hardware and open-source software tools.
• The project is suitable for both academic implementation and future smart transportation use.
• It improves road safety awareness through continuous monitoring and instant alerts.
• Overall, the system provides a complete, intelligent, and reliable road safety framework.

2.3 FEASIBILITY STUDY

The feasibility study of the project "Real-Time Accident Monitoring and Risk Prediction using AI & IoT" evaluates whether the proposed system can be successfully developed and implemented within the available resources, time, and academic scope. The project is found to be feasible in terms of technical, economic, operational, schedule, and social feasibility, as it uses affordable hardware, open-source software, and a modular development approach suitable for student-level implementation

Five key considerations involved in the feasibility analysis are:

2.3.1 Technical feasibility:

Technical feasibility refers to whether the proposed system can be developed using the available hardware, software, technologies, and technical skills. It evaluates whether the required tools and components are practical and suitable for successful implementation.

Key Technical Considerations:

- The system uses easily available hardware components such as ESP32, MPU6050 sensor, GPS module, buzzer, LCD, crash switch, and panic button.
- The software side uses Python, web technologies (HTML/CSS/JavaScript), serial communication, Telegram API, and weather APIs, which are widely supported and easy to implement.
- The AI module uses a stacked LSTM model, which is suitable for analyzing sequential sensor and environmental data.
- All required technologies are well-documented and can be integrated using standard development tools such as Arduino IDE, Python IDE, and browser-based dashboards.

Scalability:

- The system can be extended by adding more sensors such as alcohol sensors, camera modules, heart-rate sensors, or cloud storage.
- It can be scaled from a single vehicle prototype to multiple vehicles or fleet monitoring systems.

- Future upgrades can include mobile app integration, cloud dashboards, and advanced AI models.

Performance:

- The ESP32 can collect and transmit sensor data in real time.
- The Python application can process telemetry, fetch weather data, and update the dashboard with low latency.
- The LSTM model can classify accident risk effectively using sequential inputs.

2.3.2 Economic Feasibility:

Economic feasibility refers to whether the proposed system can be developed and implemented within an affordable budget, and whether the benefits of the system justify the cost of development.

Key Cost Factors:

- Low-cost hardware components:
- ESP32
- MPU6050
- GPS module
- Buzzer
- LCD
- Crash switch / Panic button

Free or open-source software:

- Python
- Machine learning libraries
- Web technologies
- Telegram Bot API
- Weather API (free tier if available)

Benefits

- Suitable for student-level MVP (Minimum Viable Product) development.
- Does not require expensive industrial hardware or paid software tools.
- Can be built with a limited academic project budget.
- Provides significant real-world value despite low cost.

Cost Benefits

- Low development cost compared to commercial safety systems.
- Reduced dependency on expensive infrastructure like advanced CCTV networks.
- Potential savings in emergency response time and reduction in accident severity.

- Can be implemented as a cost-effective smart transportation safety solution.

2.3.3 Social Feasibility

Definition

Social feasibility refers to whether the proposed system is **socially acceptable, beneficial, and useful to society**, especially in solving real-world problems that affect public welfare.

Key Social Benefits

- Addresses an important public issue: **road accidents and delayed emergency response**.
- Helps in **reducing accident severity** through early warning and faster alerts.
- Improves chances of **timely medical assistance** by sharing precise location details.
- Reduces dependence on manual reporting by witnesses or drivers.
- Supports the development of **smart transportation and public safety systems**.

Social Impact

- Can contribute to **saving human lives**.
- Improves awareness and response in accident-prone areas.
- Encourages the adoption of **AI and IoT for social good**.
- Useful for individuals, emergency responders, and smart city applications.

SYSTEM ANALYSIS AND DESIGN

3.SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Specification

Software Requirements:

Software Requirements describe the functional and non-functional capabilities that must be provided by the proposed Real-Time Accident Monitoring and Risk Prediction System. These requirements define the expected behavior of the system and ensure that it satisfies user needs, system reliability, and operational constraints. Software requirement analysis includes identifying the major system features, runtime environment, communication flow, data processing needs, and performance expectations for the real-time accident monitoring application.

The software requirements specify how the system receives real-time telemetry data from the ESP32-based IoT module, processes motion, crash, panic, and GPS inputs, and integrates live weather parameters such as rain, precipitation, wind speed, and visibility for improved risk estimation. These requirements also define how the Python-based monitoring system performs data preprocessing, applies the stacked LSTM model using a sliding window approach for risk classification, and categorizes accident risk into LOW, MEDIUM, and HIGH levels. In addition, the software must support real-time dashboard updates, local status visualization, and instant alert generation whenever a risk or accident condition is detected.

The system software must also provide support for web dashboard communication, Telegram notification services, and location-based emergency messaging using GPS coordinates and Google Maps links. It should ensure smooth serial communication between the hardware and software layers, accurate risk computation, timely display of monitoring information, and reliable transmission of alerts. These software requirements help guide the development of the monitoring, prediction, dashboard, and alert modules, ensuring proper implementation of real-time accident detection, risk prediction, and emergency communication functions.

User Requirements:

1. **Access to Monitoring System**: The user (administrator/operator) must have access to a computer or laptop to run the Python-based monitoring application.

2. **Connection to IoT Device:** The system must be connected to the ESP32-based accident monitoring module for receiving real-time telemetry data.

3. **Hardware Availability:** The user must ensure that the required hardware components are properly connected and working:

- ESP32
- MPU6050 sensor
- GPS module
- Crash switch
- Panic button
- LCD display
- Buzzer, Motor (for vehicle representation).

4. **Internet Connectivity:** The system requires internet access for:

- Fetching live weather data
- Sending Telegram alerts
- Updating the live dashboard (if applicable)

5. **System Configuration:** The user should configure required settings before running the system:

- Serial port / COM port
- Telegram Bot API credentials
- Weather API details

6. **Basic Monitoring Control:** The user should be able to:

- Start the monitoring system
- Stop the monitoring system
- Observe live dashboard output

7. **Alert Monitoring**: The user should be able to verify:

- Risk level display
- Local alerts through LCD and buzzer
- Telegram emergency notifications
- GPS location details in alerts

3.1.1 FUNCTIONAL REQUIREMENTS

Sensor Data Acquisition:

- The system should read real-time data from the MPU6050 accelerometer.
- The system should monitor crash switch and panic button status continuously.
- The system should read GPS coordinates such as latitude and longitude.

Telemetry Transmission:

- The system should transmit telemetry from ESP32 to the Python monitoring system.
- Data should be sent through serial communication in real time.
- The communication should support continuous monitoring.

Weather Data Collection:

- The system should fetch live weather conditions using a weather API.
- Weather parameters should include rain, precipitation, wind speed, and visibility.
- The weather data should be combined with sensor data for risk analysis.

Risk and Severity Computation:

- The system should process sensor and weather data for risk estimation.
- The system should classify accident risk into LOW, MEDIUM, and HIGH levels.
- The system should determine the severity level during abnormal or accident conditions.

Risk Prediction:

- The system should use a stacked LSTM model for sequence-based risk prediction.
- The system should apply a sliding window approach for analyzing sequential data.
- The system should generate a PREALERT when risk increases before a crash.

Local Alert Generation:

- The system should display alerts through the LCD display.
- The system should activate the buzzer when a risk or accident is detected.
- The system should provide immediate local warning messages.

Telegram Notification:

- The system should send Telegram emergency notifications automatically.
- Notifications should include risk/accident status and severity level.
- Notifications should include GPS location details with Google Maps link.

Live Dashboard Monitoring:

- The system should update the live dashboard in real time.
- The dashboard should display sensor values, risk level, and severity level.
- The dashboard should show GPS/location details and system status.

Emergency Trigger Handling:

- The system should support manual emergency triggering using the panic button.
- The system should treat panic activation as an emergency event.
- The system should generate alerts immediately after panic activation.

3.1.2 NON-FUNCTIONAL REQUIREMENTS:

Performance:

- The system should respond quickly during accident or high-risk conditions.
- Risk computation, local alerting, and Telegram notification should occur within a few seconds.

Reliability:

- The system should continue local alerting through LCD and buzzer even if remote communication fails.
- The system should run continuously during monitoring without frequent crashes or interruptions.

Usability:

- The dashboard should be simple, clear, and easy to read during project demonstration.
- The system should be easy to operate with minimal manual intervention.

Security:

- Configuration values such as Telegram API keys and weather API credentials should be stored securely in environment files.
- Sensitive configuration data should not be hardcoded in the program.

Maintainability:

- The system should allow easy modification of thresholds, API settings, and configuration values.
- Individual modules such as sensing, dashboard, and alerting should be easy to update independently.

Scalability:

- The system should be extendable for adding more sensors or additional monitoring features in future versions.
- It should support future expansion for multiple vehicles or cloud-based monitoring.

Compatibility:

* The system should work on standard Windows-based systems used for project demonstration.
* It should support integration with ESP32 hardware, sensors, and web dashboard technologies.

Cost Effectiveness:

- The system should be low-cost and suitable for student-level MVP development.
- The hardware and software setup should be easy to reproduce for academic demonstrations.

3.1.3 PIN CONFIGURATION

Module	ESP32 Pin
LCD RS	D15
LCD Enable	D2
LCD D4	D4
LCD D5	D16
LCD D6	D17
LCD D7	D5
Buzzer	D19
Motor	D23
MPU6050 SDA / SCL	D21 / D22
GPS TX to ESP32 RX	D26
Limit Switch	D18
Push Button	D33

3.2 FLOWCHARTS /UML DIAGRAMS

Unified Modeling Language (UML) is a general-purpose modeling language used to visualize, specify, construct, and document the components of a software system. The main aim of UML is to define a standard way to represent how a system has been designed. It acts as a blueprint for software development.

In the proposed Real-Time Accident Monitoring and Risk Prediction System, UML diagrams are used to represent the structure and working process of the overall monitoring application. Since the system integrates ESP32-based sensing, MPU6050 motion detection, GPS tracking, weather-aware risk analysis, LSTM-based risk prediction, live dashboard monitoring, and Telegram alert modules, UML diagrams help in clearly understanding the interaction between these components.

A UML diagram provides a visual representation of the system architecture and workflow, making it easier to understand how data flows from sensors to prediction and alert generation. It simplifies complex IoT communication, real-time processing, and AI-based risk analysis into an understandable graphical format for documentation, PPT, and project presentation. UML is useful in this project because it helps to clearly visualize the system structure and workflow before implementation.

It shows how components like ESP32, sensors, Python application, dashboard, and Telegram alerts interact with each other.

Using UML diagrams such as Use Case, Class, Sequence, and Activity diagrams makes the project easier to design, understand, and document. It also helps in explaining the project effectively during PPT, viva, and report presentation.

Diagrams in UML can be broadly classified as:

1. Structural Diagrams – Capture the static structure of the system.
   Structural diagrams in this project include:
   - Class Diagram
   - Component Diagram
   - Deployment Diagram

2. Behavioral Diagrams – Capture the dynamic behavior and workflow of the system.

   Behavioral diagrams in this project include:

   - Use Case Diagram 22
   - Activity Diagram
   - Sequence Diagram
   - Flow Diagram

GOALS:

The primary goals in the design of UML for the proposed Real-Time Accident Monitoring and Risk Prediction System are as follows:

- Provide a clear and expressive visual model of the IoT and AI-based accident monitoring system.
- Help developers understand the system modules and their interactions clearly.
- Represent the real-time sensor monitoring and telemetry flow effectively.
- Illustrate the risk prediction process using weather-aware analysis and LSTM-based sequence learning.
- Show the accident detection, alert generation, dashboard update, and Telegram notification flow.
- Provide extendibility for future system enhancements such as cloud integration and mobile app support.
- Maintain independence from specific programming language or implementation details.
- Support modular, scalable, and easy-to-understand system design for documentation and development.

3.2.1: BLOCK DIAGRAM

The **block diagram** of the proposed **Real-Time Accident Monitoring and Risk Prediction System** shows the interaction between the input devices, processing unit, and output modules. The **ESP32 microcontroller** acts as the central control unit of the system. It receives inputs from the **MPU6050 sensor** for motion and impact detection, the **GPS module** for location tracking, and the **panic button** for manual emergency triggering. A **power supply** is provided to operate the complete hardware setup.

Based on the received sensor data, the ESP32 processes the accident-related conditions and communicates with the monitoring system. The output devices connected to the ESP32 include the **LCD display** for showing alert messages and location details, the **buzzer** for audio alerts, and the **DC motor** for vehicle movement representation or stop indication during accident conditions. The system can also send data to the **mobile application / monitoring interface** for real-time status monitoring and emergency alert communication. Thus, the block diagram clearly represents the overall hardware flow of the accident monitoring system.

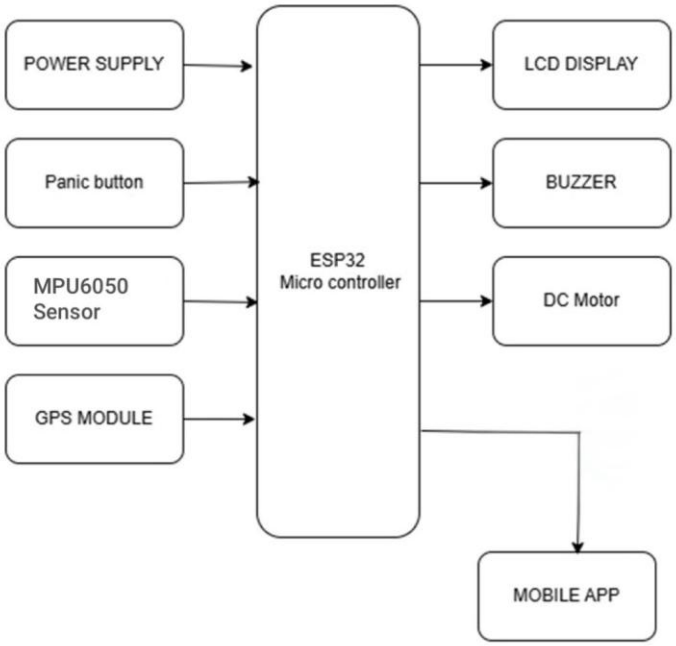


Figure 3.1: BLOCK DIAGRAM

3.2.2: USE CASE DIAGRAM

A Use Case Diagram visualizes the system's functionality from the user's perspective. It helps in understanding the interactions between the users and the system, as well as defining the system boundaries clearly. In the proposed Real-Time Accident Monitoring and Risk Prediction System, the Use Case Diagram represents the interaction between the Administrator/Operator, the Vehicle/Driver (monitored entity), and the AI & IoT-based monitoring system.

The system continuously monitors real-time sensor data such as motion, crash input, panic input, and GPS location through the ESP32-based module. It also receives weather information and performs risk prediction using the stacked LSTM model. When a high-risk or accident condition is detected, the system generates local alerts, updates the live dashboard, sends Telegram notifications, and records the event details for monitoring and response.

The administrator can monitor the live system status, view accident risk levels, observe dashboard updates, and receive emergency alerts when abnormal conditions occur. The system automatically handles sensing, risk estimation, alert generation, and status updates without continuous manual intervention. This makes the overall process efficient, intelligent, and suitable for real-time road safety support.

The main use cases represented in the diagram include:

* Monitor Sensor Data

* Fetch Weather Data

* Predict Accident Risk

* Detect Accident Condition

* Generate Local Alert

* Send Telegram Notification

* Update Live Dashboard

This use case diagram highlights the automated workflow of the system and clearly shows the interaction between monitoring, prediction, detection, and alert communication processes.

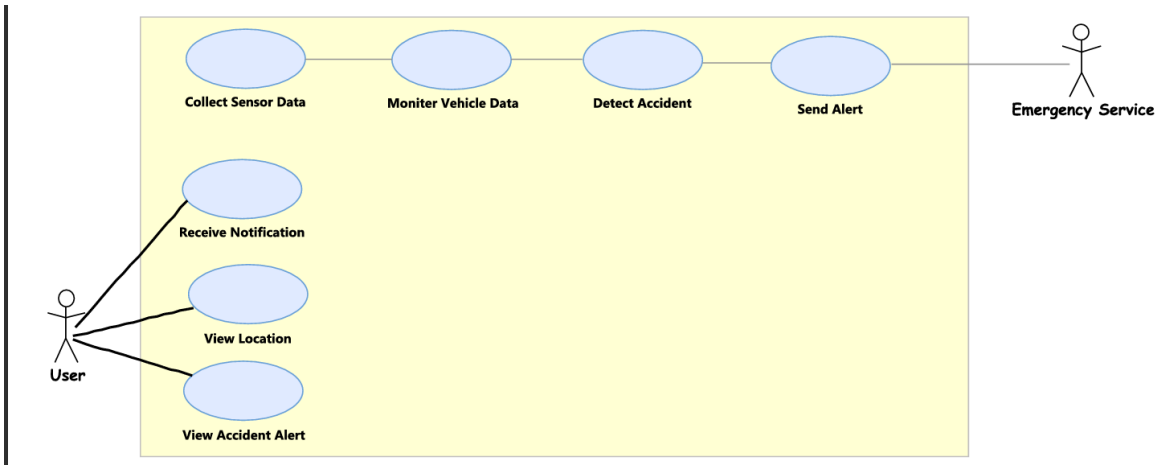


Figure 3.2: USE CASE DIAGRAM

3.2.3 CLASS DIAGRAM

A **Class Diagram** represents the **static structure** of the proposed **Real-Time Accident Monitoring and Risk Prediction System** by showing the main classes and their relationships. The major classes include **System Controller, Sensor Manager, Weather Service, Risk Predictor, Alert Manager, Dashboard Manager, and Data Logger**.

The **System Controller** manages the overall workflow of the system. The **Sensor Manager** collects telemetry data from the **ESP32 module**, including **MPU6050, crash switch, panic button, and GPS**. The **Weather Service** fetches live weather data, and the **Risk Predictor** uses the **stacked LSTM model** to estimate accident risk levels.

The **Alert Manager** handles **LCD, buzzer, and Telegram alerts**, while the **Dashboard Manager** updates the **live web dashboard**. The **Data Logger** stores important events such as **risk changes, crash detections, and alerts**. Thus, the class diagram shows the **modular structure** of the system for monitoring, prediction, alerting, and data management.

The class diagram also helps in understanding how each module performs a specific role while working together as a complete system. It clearly represents the interaction between sensing, weather analysis, risk prediction, alert generation, dashboard monitoring, and data logging modules. This improves system clarity, supports modular development, and makes the design easier to implement, test, and maintain.

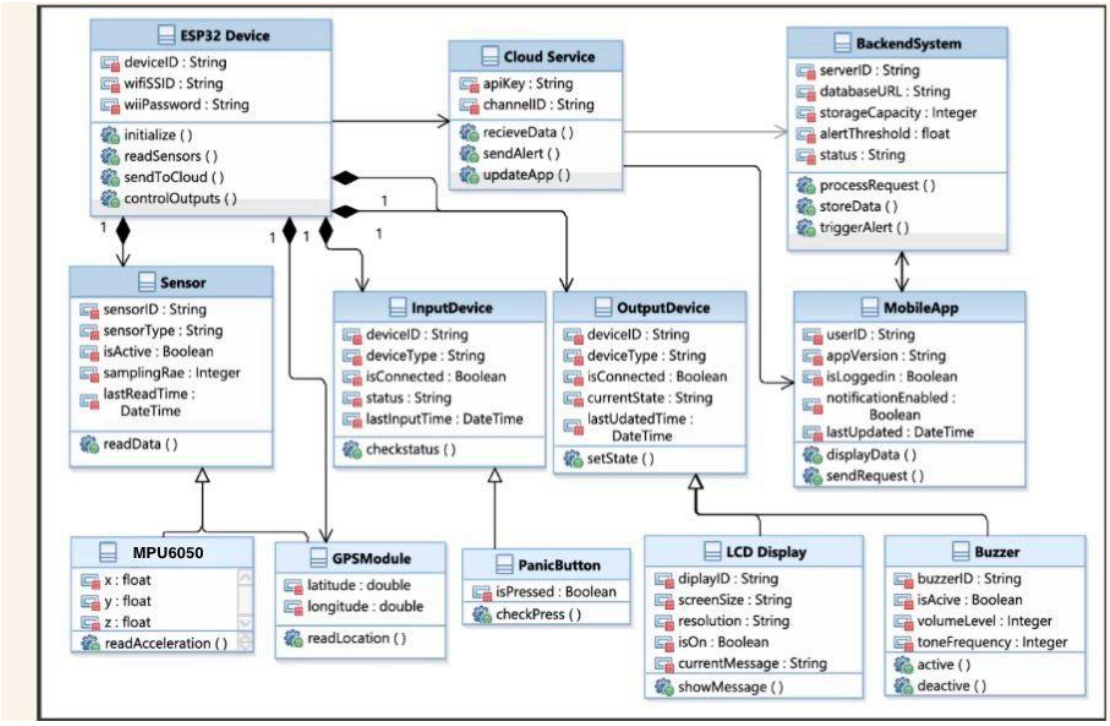


Figure 3.3: CLASS DIAGRAM

3.2.4 ACTIVITY DIAGRAM

The **Activity Diagram** represents the workflow of the proposed **Real-Time Accident Monitoring and Risk Prediction System**. It shows how the system processes real-time sensor data and handles monitoring, risk prediction, and alert generation. The process begins with **system initialization** and activation of the **ESP32-based IoT module**. The system continuously reads data from the **MPU6050 sensor, crash switch, panic button, and GPS module**, and sends the telemetry to the **Python-based monitoring system**.

The Python system receives the sensor data and fetches **live weather parameters** such as **rain, precipitation, wind speed, and visibility**. The collected data is then pre-processed and passed to the **stacked LSTM model** using a **sliding window approach** for accident risk prediction. If the predicted risk is **LOW** and no crash or panic condition is detected, the system continues monitoring. If the risk becomes **MEDIUM or HIGH**, or if a **crash or panic event** is detected, the system moves to the alert process.

During the alert process, the system computes the **severity level** and provides **local alerts through the LCD and buzzer**. At the same time, it updates the **live web dashboard** with current sensor values, risk level, severity level, and GPS details. It also sends a **Telegram emergency notification** containing the **alert status, severity level, and Google Maps location link**. After completing the alert actions, the system returns to the monitoring state and continues processing incoming data. This activity diagram clearly illustrates the **decision flow and continuous monitoring process** of the system.

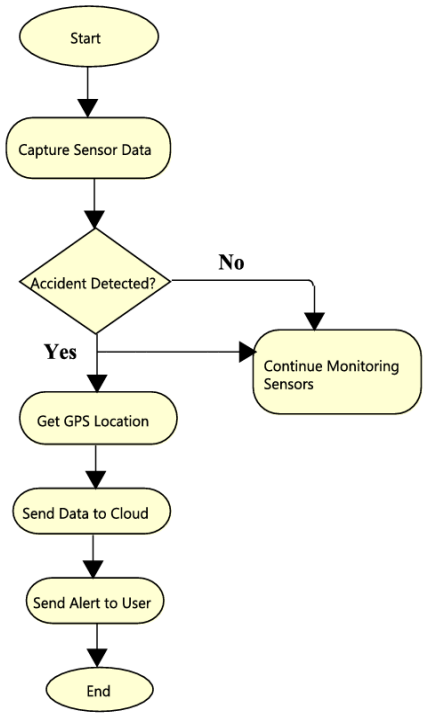


Figure 3.4: ACTIVITY DIAGRAM

3.2.5 STATE MACHINE DIAGRAM

The **State Machine Diagram** represents the different states of the proposed **Real-Time Accident Monitoring and Risk Prediction System** and the transitions between them during operation. It shows how the system moves from one state to another based on sensor inputs, risk levels, and alert conditions. The system starts in the **Initialization State**, where the **ESP32 module, sensors, Python monitoring system, and communication services** are activated.

After initialization, the system enters the **Monitoring State**, where it continuously reads data from the **MPU6050 sensor, crash switch, panic button, and GPS module**. At the same time, the Python system fetches **live weather data** and performs **risk prediction** using the **stacked LSTM model**. If the predicted risk is **LOW** and no crash or panic event is detected, the system remains in the monitoring state. If the risk becomes **MEDIUM or HIGH**, the system moves to the **Risk Alert State**. If a **crash or panic event** is detected, the system directly enters the **Emergency Alert State**.

In the **Risk Alert State**, the system generates warning alerts and updates the dashboard. In the **Emergency Alert State**, the system activates the **LCD and buzzer**, updates the **live dashboard**, and sends a **Telegram emergency notification** with **severity level and Google Maps location details**. After completing the alert process, the system returns to the **Monitoring State** and continues operation. Thus, the state machine diagram clearly shows the **state transitions between initialization, monitoring, risk detection, emergency alerting, and return to monitoring**.

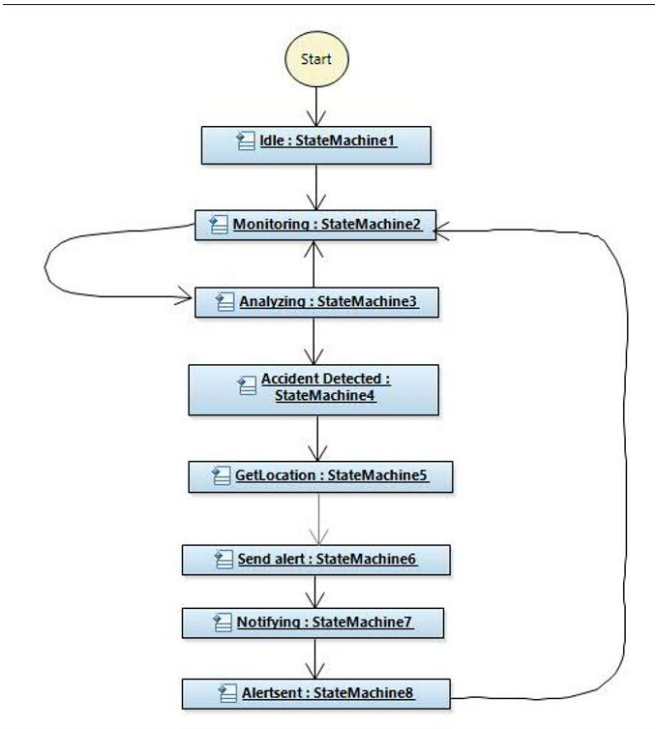


Figure 3.5: STATE MACHINE DIAGRAM

3.2.6 SEQUENCE DIAGRAM

A **Sequence Diagram** represents the order of interactions between system components over time. The sequence diagram shows the interaction between the **ESP32 Module, Python Monitoring System, Weather API, LSTM Risk Predictor, Alert Manager, Dashboard Module, and Telegram Service**. The process begins when the **ESP32 module** collects data from the **MPU6050 sensor, crash switch, panic button, and GPS module**, and sends the telemetry to the **Python monitoring system** through serial communication.

The Python monitoring system receives the telemetry and requests **live weather data** from the **Weather API**. After collecting both sensor and environmental data, the system preprocesses the data and sends it to the **LSTM Risk Predictor** for sequence-based risk analysis. The model processes the data using a **sliding window approach** and returns the predicted **risk level** such as **LOW, MEDIUM, or HIGH**. The monitoring system then checks whether a **high-risk, crash, or panic condition** exists.

If an abnormal condition is detected, the system sends the status to the **Alert Manager**, which activates **local alerts through the LCD and buzzer**. At the same time, the monitoring system updates the **Dashboard Module** with live sensor values, risk level, severity level, and GPS details. It also sends alert details to the **Telegram Service**, which delivers an emergency notification containing the **severity level and Google Maps location link**. After completing these actions, the system continues monitoring incoming data. This sequence diagram clearly illustrates the **real-time interaction between sensing, risk prediction, dashboard updating, and alert communication modules**.

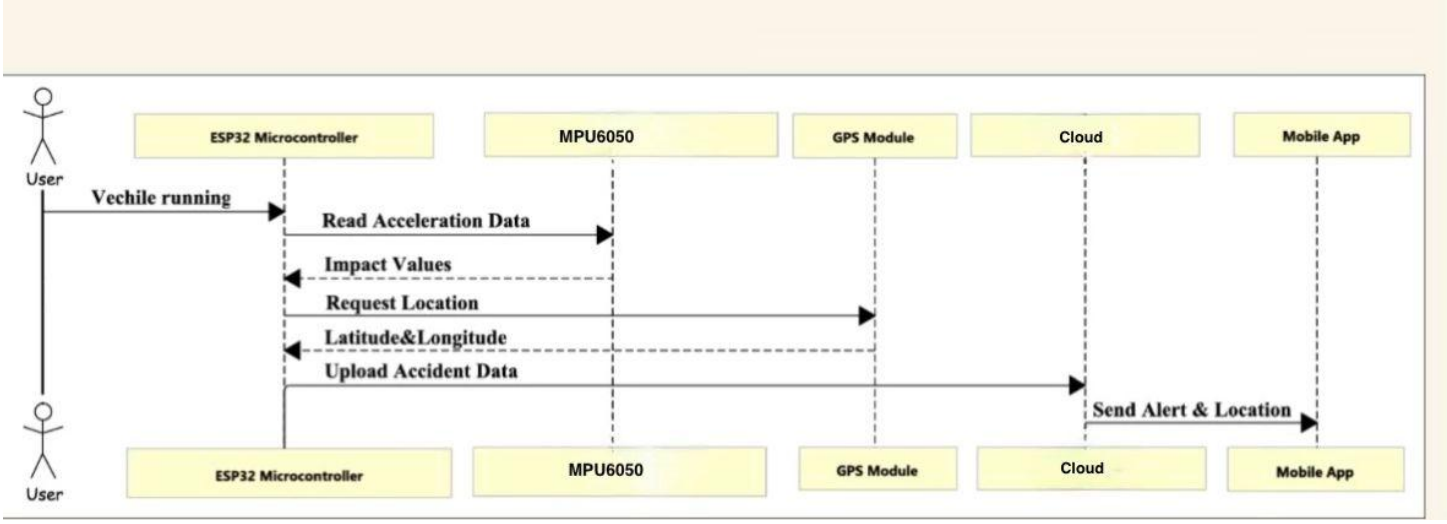


Figure 3.6: SEQUENCE DIAGRAM

3.2.7 COMPONENT DIAGRAM

A **Component Diagram** illustrates how different software and hardware components interact within a system. It helps in understanding the **modular structure** and dependencies between various components of the proposed **Real-Time Accident Monitoring and Risk Prediction System**. The component diagram represents the major components involved in **sensor data collection, risk prediction, dashboard monitoring, and alert generation**.

The **IoT Sensing Module** includes the **ESP32 microcontroller, MPU6050 sensor, GPS module, crash switch, panic button, LCD display, buzzer, and motor for vehicle representation**. This module is responsible for collecting real-time motion, crash, panic, and location data and providing local alert indications. The **Monitoring and Processing Module** includes the **Python-based monitoring system**, which receives telemetry from the ESP32, preprocesses data, and coordinates the complete workflow. It acts as the main controller of the system.

The **Risk Prediction Module** includes the **stacked LSTM model**, which analyses sequential sensor and environmental data using a **sliding window approach** and classifies accident risk into **LOW, MEDIUM, and HIGH** levels. The **Weather Integration Module** fetches live weather data such as **rain, precipitation, wind speed, and visibility** using an API for weather-aware risk estimation. The **Dashboard Module** displays live sensor values, risk level, severity level, and GPS details for real-time monitoring. The **Alert Module** handles **Telegram notifications** and local alerts through the **LCD and buzzer**, while external components include the **Weather API** and **Telegram Service**. The **Main Controller** coordinates all modules and manages communication between them. This component diagram clearly illustrates the **modular interaction and system architecture** of the proposed solution.

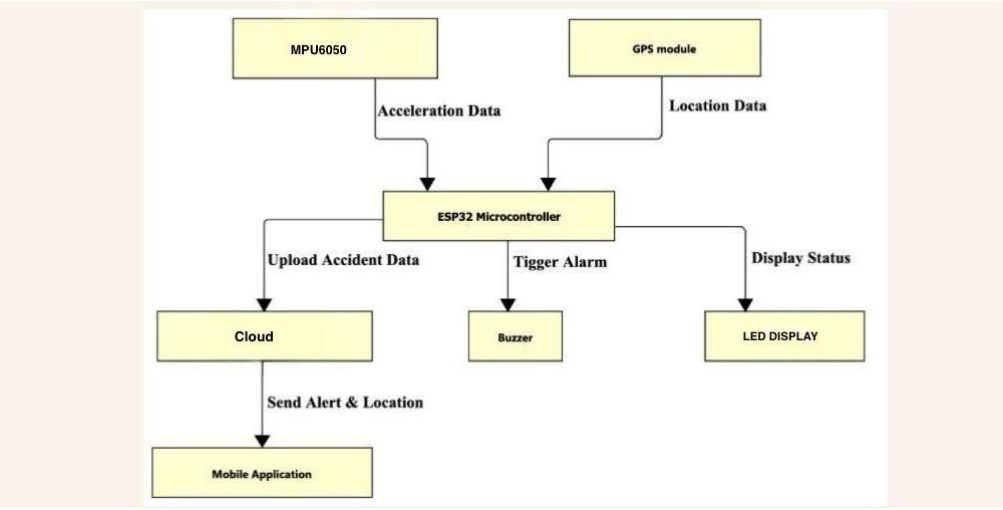


Figure 3.7: COMPONENT DIAGRAM

3.2.8 DEPLOYMENT DIAGRAM

The **Deployment Diagram** represents the physical architecture of the system and shows how the software components are deployed on hardware devices. The proposed **Real-Time Accident Monitoring and Risk Prediction System** consists of three main nodes: **ESP32 IoT Module, Monitoring Computer/Laptop, and External Services**. The **ESP32 IoT module** collects real-time data from the **MPU6050 sensor, GPS module, crash switch, and panic button**, and also controls local output devices such as the **LCD display, buzzer, and motor**.

The **Monitoring Computer/Laptop** runs the **Python-based monitoring application**, which receives telemetry from the ESP32 through **serial communication**. This node also runs the **stacked LSTM model**, fetches **live weather data**, computes **risk and severity levels**, updates the **live web dashboard**, and manages the complete monitoring workflow. It acts as the main processing unit of the system.

The **External Services** include the **Weather API** and **Telegram service**, which are connected through the internet. The weather service provides live parameters such as **rain, precipitation, wind speed, and visibility** for weather-aware risk analysis. The Telegram service is used to send emergency notifications containing **severity level and Google Maps location link**. The deployment diagram shows the communication between the **local hardware system**, the **monitoring computer**, and the **external online services**, clearly illustrating the physical deployment and interaction of the proposed system.

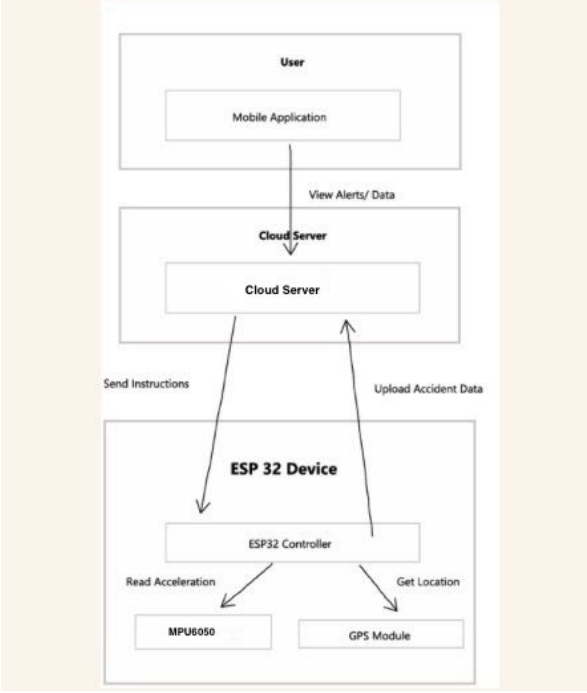


Figure 3.8: DEPLOYMENT DIAGRAM

3.2.9 FLOW DIAGRAM

The **Flow Diagram** represents the overall working process of the proposed **Real-Time Accident Monitoring and Risk Prediction System**. The system starts and initializes the **ESP32-based IoT module**, Python monitoring system, and all required detection and alert modules. The ESP32 continuously collects data from the **MPU6050 sensor, crash switch, panic button, and GPS module** and sends the telemetry to the Python system. The Python application also fetches **live weather data** for risk analysis.

The system then preprocesses the collected sensor and weather data and passes it to the **stacked LSTM model** for accident risk prediction. If the predicted risk is **LOW** and no crash or panic condition is detected, the system continues monitoring. If the risk becomes **MEDIUM or HIGH**, or if a **crash or panic event** is detected, the system calculates the **severity level** and triggers the alert process.

During the alert process, the system provides **local alerts through the LCD and buzzer**, updates the **live dashboard**, and sends a **Telegram emergency notification** with **severity level and Google Maps location details**. After completing these actions, the system continues monitoring incoming data. Thus, the flow diagram summarizes the complete working pipeline of the proposed system from **data collection to prediction and alert generation**.

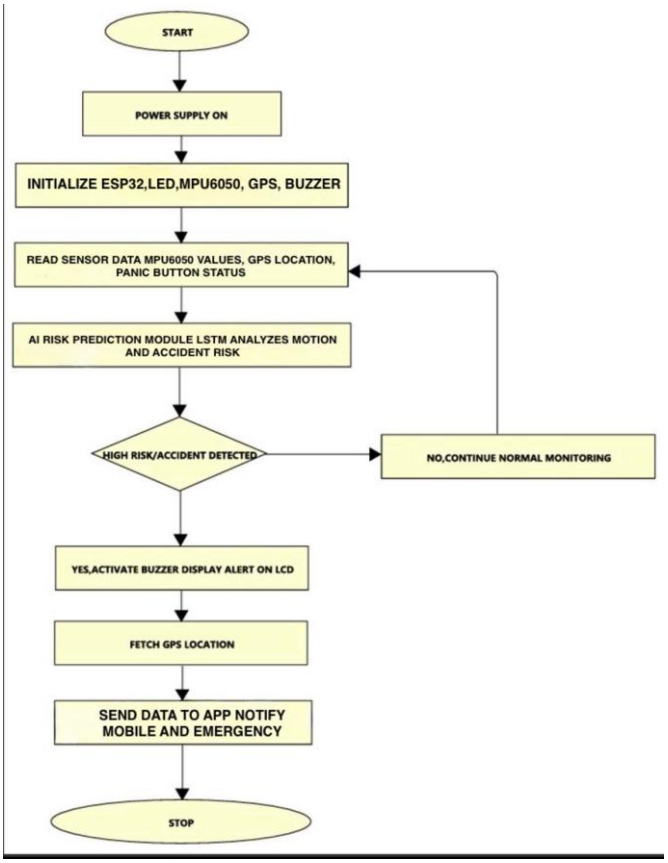


Figure 3.9: FLOW DIAGRAM

3.3 DESIGN AND TEST STEPS / CRITERIA

Design Steps:

Step 1: Environment Setup

Install Python and required libraries such as **pyserial, pandas, NumPy, TensorFlow/keras, requests, flask (if used), and dotenv**.

Step 2: Hardware Setup

Connect and configure the **ESP32**, **MPU6050 sensor**, **GPS module**, **crash switch**, **panic button**, **LCD display**, **buzzer**, and **motor** for vehicle representation.

Step 3: Serial Communication Setup

Establish serial communication between the **ESP32** and the **Python-based monitoring system** for real-time telemetry transfer.

Step 4: Data Collection Setup

Configure the ESP32 to read **motion data, crash switch status, panic input, and GPS location** continuously and send the data to the monitoring system.

Step 5: Weather API Integration

Integrate the **weather API** to fetch live weather parameters such as **rain, precipitation, wind speed, and visibility**.

Step 6: Data Preprocessing

Preprocess sensor and weather data for analysis and arrange the input using a **sliding window approach** for sequence-based prediction.

Step 7: Model Initialization

Load the trained **stacked LSTM model** for accident risk prediction.

Step 8: Risk Prediction

Use the LSTM model to classify accident risk into **LOW, MEDIUM, and HIGH** levels based on sensor and weather data.

Step 9: Severity Computation

Compute the **severity level** when a crash, panic, or abnormal condition is detected.

Step 10: Local Alert Generation

Trigger **LCD display alerts** and **buzzer notifications** during high-risk or accident conditions.

Step 11: Dashboard Update

Update the **live web dashboard** with sensor values, risk level, severity level, and GPS location details in real time.

Step 12: Telegram Notification

Send **Telegram emergency alerts** with **risk/accident status, severity level, and Google Maps location link**.

Step 13: Logging and Monitoring

Store event details such as **risk status, crash/panic events, and alert history** for monitoring and analysis.

Test Criteria:

- Sensor data acquisition accuracy
- Serial communication reliability
- Weather API data retrieval verification
- LSTM risk prediction accuracy
- Risk level classification validation
- Severity level computation verification
- LCD and buzzer alert functionality
- Telegram alert generation verification
- Live dashboard update performance
- GPS location accuracy
- System response time during accident conditions
- Continuous monitoring stability

3.3.1 Implementation and Setup Details

A. System Prerequisites and Downloads

Before executing the proposed **Real-Time Accident Monitoring and Risk Prediction System**, the host system must be properly configured with the required software tools, libraries, and hardware interfaces. These prerequisites ensure smooth communication between the **ESP32 hardware module** and the **Python-based**

monitoring application, support machine learning model execution, and enable dashboard visualization and Telegram alert generation.

Python Version (3.10 Recommended):

Python serves as the **primary programming environment** for the monitoring, prediction, dashboard, and alert modules of the system. **Python 3.10** is recommended because it provides good compatibility with major libraries such as **TensorFlow, NumPy, Pandas, PySerial, Requests, and Flask**. Using unsupported or newer versions may lead to dependency conflicts or runtime issues during model loading and API integration.

Download Link:
https://www.python.org/downloads/release/python-3100/

Arduino IDE (for ESP32 Programming):

The **Arduino IDE** is required to program and upload code into the **ESP32 microcontroller**, which acts as the core IoT controller of the system. It is used to configure sensor reading, GPS data acquisition, crash switch handling, panic button monitoring, and serial telemetry transmission. The **ESP32 board package** must be installed in the Arduino IDE before uploading the firmware.

Download Link:
https://www.arduino.cc/en/software

ESP32 Board Package:

To enable ESP32 programming inside Arduino IDE, the **ESP32 board definitions** must be installed using the **Boards Manager**. This package provides support for compiling and uploading code to the ESP32 microcontroller. Without this package, the Arduino IDE will not recognize ESP32 boards.

Board Manager URL:
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

PySerial Library:

The system uses **serial communication** between the **ESP32 module** and the **Python monitoring application**. The **PySerial** library is required in Python to receive telemetry data such as accelerometer values, crash switch state, panic input, and GPS coordinates. It ensures reliable real-time data transfer from hardware to software.

Install Command:

pip install pyserial

TensorFlow / Keras:

The **stacked LSTM model** used for accident risk prediction is implemented using **TensorFlow/Keras**. These libraries are required for loading the trained model, processing sequential sensor and weather data, and classifying accident risk into **LOW, MEDIUM, and HIGH** levels. Proper installation ensures smooth execution of the AI-based prediction module.

Install Command:

pip install tensorflow

NumPy and Pandas:

The system uses **NumPy** and **Pandas** for data preprocessing, numerical operations, and structured handling of sensor and weather input data. These libraries help in preparing the data for the **sliding window sequence generation** required by the LSTM model and for maintaining logs or structured records if needed.

Install Command:

pip install numpy pandas

Requests Library (Weather API Integration):

The **Requests** library is used to fetch **live weather parameters** such as **rain, precipitation, wind speed, and visibility** from the weather API. These environmental inputs are combined with sensor telemetry to perform **weather-aware accident risk estimation**. A stable internet connection is required for this module.

Install Command:

pip install requests

Flask / Web Dashboard Dependencies (if used):

The **live dashboard** of the system is built using **HTML, CSS, JavaScript, and JSON**, and if a Python backend is used, **Flask** can be used to serve real-time monitoring data. This allows the dashboard to display sensor values, risk score, severity level, weather data, and GPS location dynamically during execution.

Install Command:

pip install flask

Telegram Bot Setup:

The system uses **Telegram Bot API** for sending emergency alerts automatically. A Telegram bot must be created using **BotFather**, and the generated **Bot Token** and **Chat ID** must be configured in the Python application. This module is responsible for sending **severity level, alert status, and Google Maps location link** during warning or accident conditions.

Bot Setup Link:

https://core.telegram.org/bots

Environment Variables (.env Configuration):

Sensitive configuration values such as **Telegram Bot Token, Chat ID, Weather API Key, and Serial Port settings** should be stored securely in a **.env file**. This prevents hardcoding of credentials in the source code and improves maintainability and security of the project.

Required Values Example:

- Telegram Bot Token
- Telegram Chat ID
- Weather API Key
- Serial COM Port

Web Browser (Dashboard Monitoring):

A modern web browser such as **Google Chrome** or **Microsoft Edge** is required to open and monitor the **live dashboard interface**. The dashboard displays real-time values such as **accelerometer data, weather conditions, risk score, severity level, and GPS location**.

Download Links:

https://www.google.com/chrome/

https://www.microsoft.com/edge

B. Hardware Setup Requirements

The following hardware components must be properly connected and powered before running the system:

- **ESP32 Microcontroller**
- **MPU6050 Accelerometer / Gyroscope Sensor**
- **GPS Module**
- **Crash / Limit Switch**
- **Panic Button**
- **LCD Display**
- **Buzzer**
- **Motor (for vehicle representation)**
- **USB Cable / Power Supply**

C. Initial Setup Procedure

Step 1: Install Python 3.10 and required Python libraries

Step 2: Install Arduino IDE and configure ESP32 board support

Step 3: Connect ESP32 and upload firmware code

Step 4: Connect MPU6050, GPS, crash switch, panic button, LCD, buzzer, and motor

Step 5: Configure serial port / COM port in Python application

Step 6: Add Telegram Bot Token, Chat ID, and Weather API Key in .env file

Step 7: Load the trained stacked LSTM model into the Python system

Step 8: Start the Python monitoring application

Step 9: Open the live dashboard in browser

Step 10: Verify telemetry, risk prediction, alerts, and dashboard updates

D. Purpose of Setup

This setup ensures that the system can:

- Read real-time sensor data from the **ESP32 module**
- Fetch live weather data for **weather-aware risk analysis**
- Run the **LSTM-based risk prediction model**

• Update the **live dashboard** in real time

• Send **Telegram emergency alerts** with GPS location

• Provide **local alerts** through LCD and buzzer

3.4 ALGORITHMS AND PSEUDO CODE

3.4.1 Sensor Data Acquisition and Telemetry Transmission Algorithm

Step 1: Start

Step 2: Initialize ESP32 and connected sensors

Step 3: Read MPU6050 accelerometer data

Step 4: Read crash switch status

Step 5: Read panic button status

Step 6: Read GPS coordinates

Step 7: Send telemetry data to Python system through serial communication

Step 8: Continue monitoring

Step 9: Stop

Pseudo Code:

```
START
Initialize ESP32
Initialize MPU6050, Crash Switch, Panic Button, GPS
WHILE True DO
   Read Motion Data
   Read Crash Switch Status
   Read Panic Button Status
   Read GPS Data
   Send Telemetry to Python System
END WHILE
STOP
```

3.4.2 Risk Prediction and Alert Generation Algorithm

Step 1: Start

Step 2: Receive telemetry data from ESP32

Step 3: Fetch live weather data

Step 4: Preprocess sensor and weather data

Step 5: Apply sliding window for sequence preparation

Step 6: Load stacked LSTM model

Step 7: Predict accident risk level

Step 8: Check crash or panic condition

Step 9: Compute severity level

Step 10: Generate local alert through LCD and buzzer

Step 11: Update live dashboard

Step 12: Send Telegram notification

Step 13: Continue monitoring

Step 14: Stop

Pseudo Code:

```
START
Receive Telemetry Data from ESP32
Load LSTM Model
WHILE True DO
   Fetch Weather Data
   Preprocess Sensor and Weather Data
   Apply Sliding Window
   Predict Risk Level using LSTM
   IF Risk = MEDIUM or HIGH OR Crash Detected OR Panic Pressed THEN
      Compute Severity Level
      Activate LCD Alert
      Activate Buzzer
```

Update Dashboard

Send Telegram Notification

END IF

END WHILE

STOP

3.5: TESTING PROCESS

Testing is a crucial phase in the development of the proposed **Real-Time Accident Monitoring and Risk Prediction System**. The main objective of testing is to ensure that the system performs **hardware sensing, serial communication, risk evaluation, alert generation, dashboard updating, and controlled failure handling** accurately and without errors. Testing helps in identifying bugs, validating the reliability of the **ESP32-based sensing module**, and ensuring proper integration of the **Python monitoring application, risk logic, dashboard interface, and Telegram alert system**. The testing strategy covers multiple levels such as **unit testing, integration testing, functional testing, and reliability testing**. Both internal logic validation and practical runtime behaviour are considered to verify the overall system performance. The system is tested using **real-time sensor inputs, crash trigger conditions, warning scenarios, and communication failure conditions** to ensure reliability and stability during demonstration and operation.

3.5.1 Types of Testing

Different types of testing are performed to verify the integrity, reliability, and correctness of the proposed **Real-Time Accident Monitoring and Risk Prediction System**.

3.5.1.1 Functional Testing

Functional testing ensures that the system performs the intended operations correctly. The following functionalities are tested:

- Accident trigger verification
- Warning trigger verification
- LCD display operation
- Buzzer activation
- Motor stop functionality
- Alert flow validation

This testing verifies that each feature works according to the system requirements and that the system responds correctly during **warning and accident conditions**.

3.5.1.2 Performance Testing

Performance testing evaluates system behaviour under continuous monitoring and real-time accident conditions. This includes:

• Real-time response of hardware sensing
• Speed of serial telemetry transfer from ESP32 to Python
• Risk logic execution time
• Dashboard update performance
• Telegram alert generation delay
• System response during accident and warning triggers

The system is tested to ensure that monitoring continues without lag while alerts and updates are generated.

3.5.1.3 Security Testing

Security testing ensures safe handling of credentials and configuration data. This involves:

• Secure storage of **Telegram API credentials**
• Protection of configuration values using **environment files (.env)**
• Avoiding hardcoding of sensitive keys in source code
• Safe handling of dashboard and alert configuration data
• Preventing unauthorized exposure of API settings

This ensures that sensitive information remains protected during development and deployment.

3.5.1.4 Usability Testing

Usability testing ensures that the system is easy to operate and monitor during real-time usage and project demonstration.

• Checking dashboard readability and clarity
• Verifying proper display of alert status, risk level, and severity
• Confirming LCD message visibility

• Ensuring simple start/stop operation
• Monitoring system output clearly during demo

This testing ensures that the system can be easily understood and operated by the administrator or project evaluator.

3.5.1.5 Regression Testing

Regression testing ensures that updates or modifications do not affect existing functionality.

• Testing after changes in risk logic or severity mapping
• Testing after dashboard update modifications
• Testing after Telegram alert integration changes
• Testing after ESP32 telemetry format changes
• Ensuring no unexpected changes in system behaviour

This helps maintain system stability and ensures that previous features continue to work correctly after updates.

3.5.1.6 System Testing

System testing verifies the complete workflow of the proposed application.

• Sensor acquisition → ESP32 telemetry generation
• ESP32 telemetry → Python parsing
• Python parsing → Risk logic execution
• Risk logic → LCD / buzzer / motor control
• Risk logic → Dashboard update
• Dashboard update → Telegram alert generation
• Failure condition → Controlled failure handling

This testing ensures that the complete end-to-end system operates correctly as an integrated solution.

3.5.1.6.1 White Box Testing

White box testing is applied to verify internal logic and program flow. The source code is tested to ensure that all conditions, loops, and decision branches function correctly.

Objectives:

• Validate sensor acquisition logic
• Test crash switch and button state handling
• Verify severity mapping conditions
• Check telemetry parsing and JSON handling
• Test alert and dashboard update logic

Manual code inspection, debugging tools, and runtime validation are used to verify internal operations.

3.5.1.6.2 Black Box Testing

Black box testing is performed without considering the internal code logic. The system is tested using real-time inputs and practical scenarios.

Scenarios Tested:

• Normal monitoring condition
• Warning trigger condition
• Accident trigger condition
• Crash switch activation
• Panic button activation
• GPS unavailable condition
• Telegram / internet failure condition

The system output is verified for each scenario to ensure correct external behaviour.

3.5.2 Levels of Testing

3.5.2.1 Unit Testing

Unit testing verifies individual modules independently.

Test Objectives:

• Sensor acquisition verification
• Button state validation
• Crash switch operation
• Severity level mapping

Each module is tested separately before full system integration.

3.5.2.2 Integration Testing

Integration testing verifies interaction between connected modules.

Test Scenarios:

• ESP32 JSON telemetry + Python parsing
• Python parsing + command acknowledgement
• Command acknowledgement + dashboard JSON update
• Dashboard update + Telegram API integration

Result:

Modules communicate correctly without errors and data flows properly across the system.

3.5.2.3 Acceptance Testing

Acceptance testing ensures that the system meets project requirements and performs as expected in real-time conditions.

Test Conditions:

• Real-time sensor monitoring
• Warning trigger generation
• Accident trigger generation
• LCD and buzzer alert activation
• Motor stop operation
• Dashboard update verification
• Telegram alert received
• Controlled failure handling

Result:

The system successfully performs **real-time accident monitoring, risk logic execution, local alert generation, dashboard updating, Telegram notification, and controlled failure handling** according to the project requirements.

TEST CASES:

Test No	01
Test Name	Startup and Telemetry
Test Scenario	Run ESP32 and Python monitor
Expected Output	Telemetry received and dashboard NORMAL
Actual Output	System initialized successfully
Remarks	Success

TABLE -1: TEST CASE -1

Test No	02
Test Name	Crash Switch - LOW
Test Scenario	Press crash switch
Expected Output	LOW severity accident alert
Actual Output	LOW severity alert
Remarks	Success

TABLE -2: TEST CASE -2

Test No	03
Test Name	Sudden Motion - MEDIUM
Test Scenario	Apply quick jerk to MPU6050
Expected Output	MEDIUM severity alert
Actual Output	MEDIUM severity detected
Remarks	Success

TABLE-3: TEST CASE-3

Test No	04
Test Name	Vigorous Shake – HIGH
Test Scenario	Shake MPU6050 vigorously
Expected Output	HIGH severity accident alert
Actual Output	HIGH severity alert generated
Remarks	Success

TABLE-4: TEST CASE-4

Test No	05
Test Name	GPS/Telegram Failure
Test Scenario	No GPS fix or internet failure
Expected Output	Local alert continues without crash
Actual Output	Failure handled safely
Remarks	Failure handled correctly

TABLE-5: TEST CASE-5

RESULTS / OUTPUTS

4. RESULTS / OUTPUTS

The output section includes LCD status, dashboard status, Python terminal output, Telegram alerts, hardware setup, training curve, confusion matrix, and performance metrics.

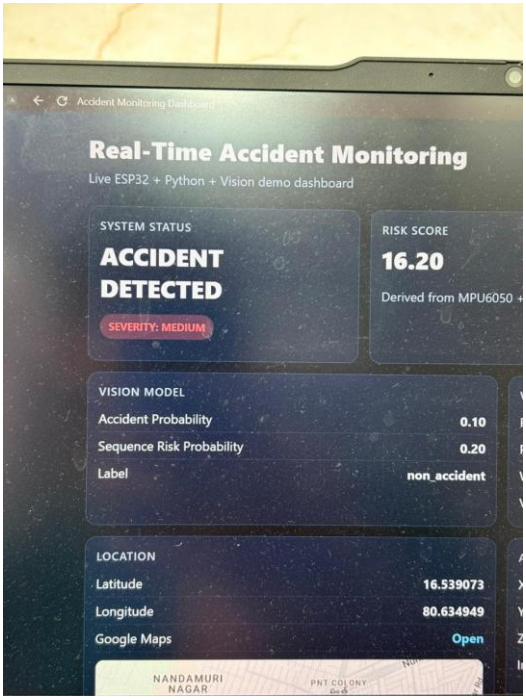


Figure 4.1: Dashboard output showing real-time accident detection with severity level and location tracking.

```
Accelerometer Data
X-axis: -0.02
Y-axis: -0.74
Z-axis: 0.65

Location
Latitude : 16.5062
Longitude: 80.6480

Vision Model
Accident Prob : 0.10
Sequence Risk : 0.20
Label         : non_accident

Status: NORMAL
--------------------------------------

------ Accident Monitoring System ------

Accelerometer Data
X-axis: -0.02
Y-axis: -0.75
Z-axis: 0.64

Location
Latitude : 16.5062
Longitude: 80.6480

Vision Model
Accident Prob : 0.10
Sequence Risk : 0.20
Label         : non_accident

Status: NORMAL
--------------------------------------
```

Figure 4.2: Console output showing real-time sensor data, GPS location, risk analysis, and system status in a normal condition

```
Vision Model
Accident Prob : 0.10
Sequence Risk : 0.20
Label         : non_accident

Status: ACCIDENT DETECTED
----------------------------------------
Sending alert to emergency contacts...

Google Maps Location:
https://maps.google.com/?q=16.5391,80.6348

------ Accident Monitoring System ------

Accelerometer Data
X-axis: -0.08
Y-axis: -0.56
Z-axis: 0.77

Location
Latitude : 16.5391
Longitude: 80.6347

Severity: LOW

Vision Model
Accident Prob : 0.10
Sequence Risk : 0.20
Label         : non_accident

Status: ACCIDENT DETECTED
----------------------------------------
Sending alert to emergency contacts...

Google Maps Location:
https://maps.google.com/?q=16.5391,80.6347
```

Figure 4.3: Console output showing real-time sensor data, GPS location, risk analysis, and system status showing accident condition

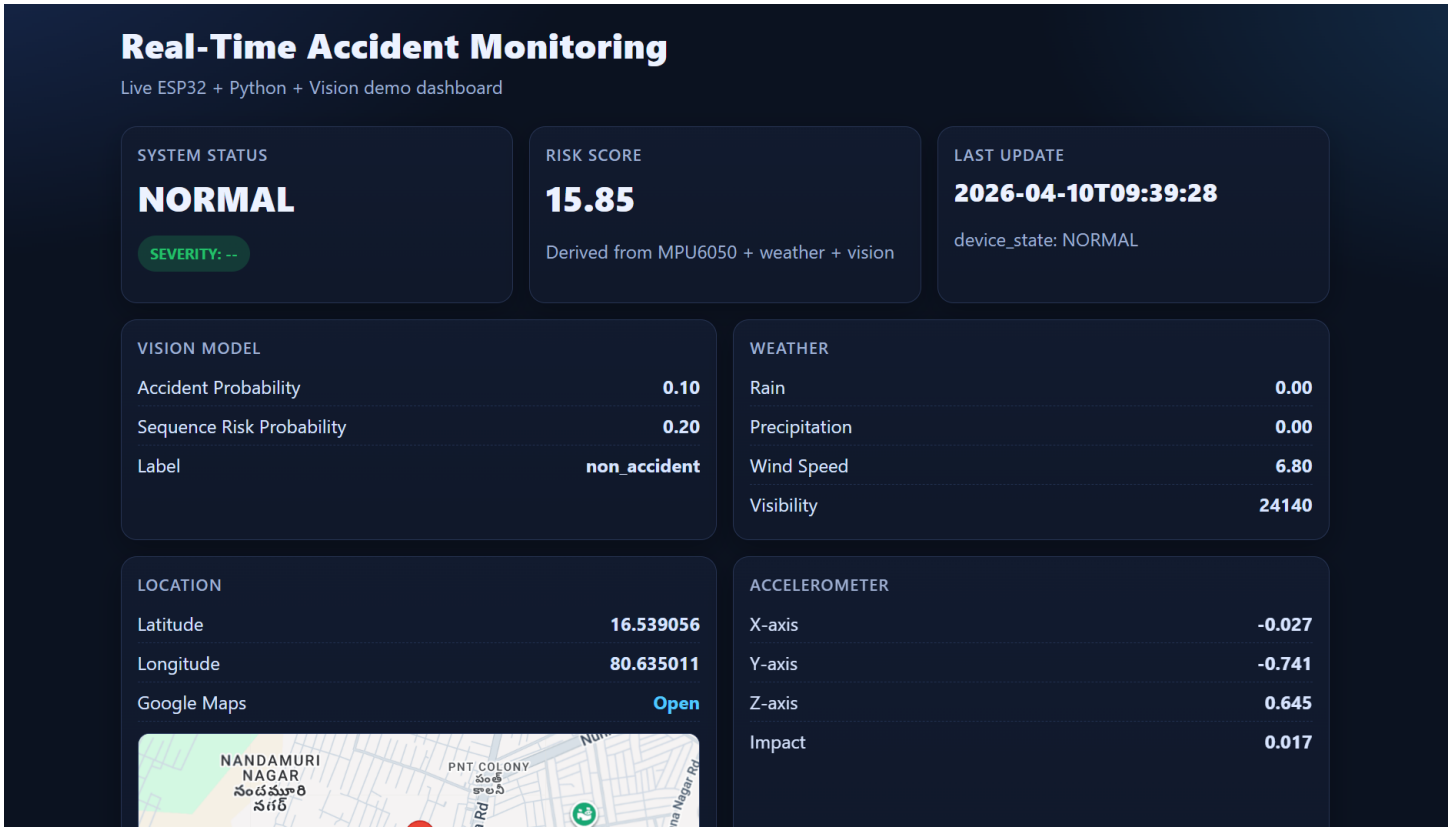


Figure 4.4: Dashboard output showing normal status, real-time sensor data, weather conditions, and location tracking.

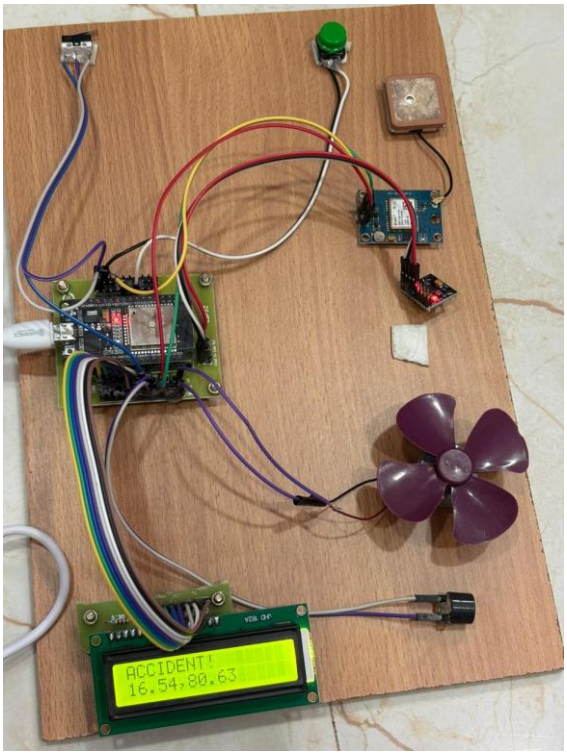


Figure 4.4:Hardware Prototype of Real-Time Accident Monitoring System showing accident status

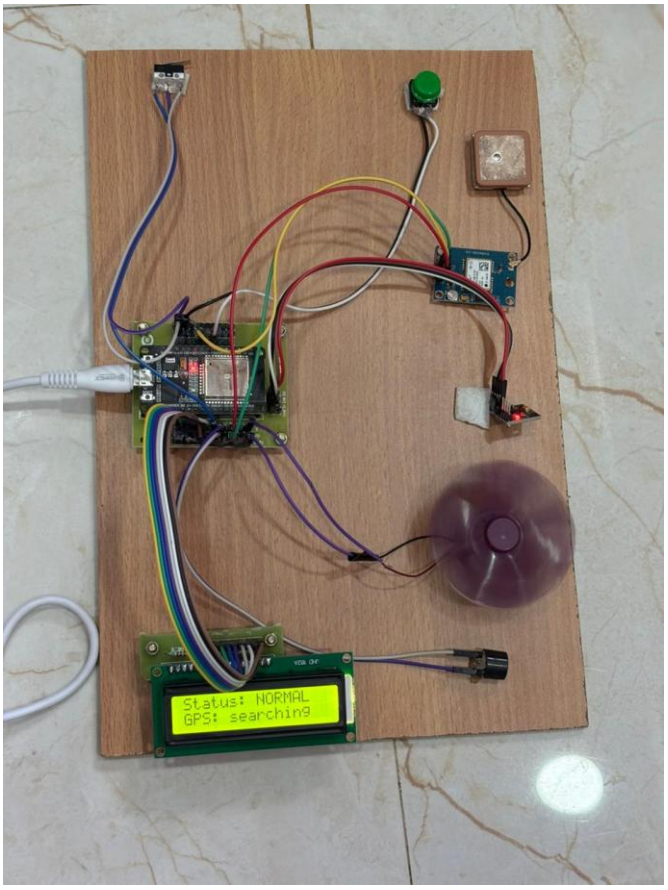


Figure 4.5: Hardware Prototype of Real-Time Accident Monitoring System showing normal status

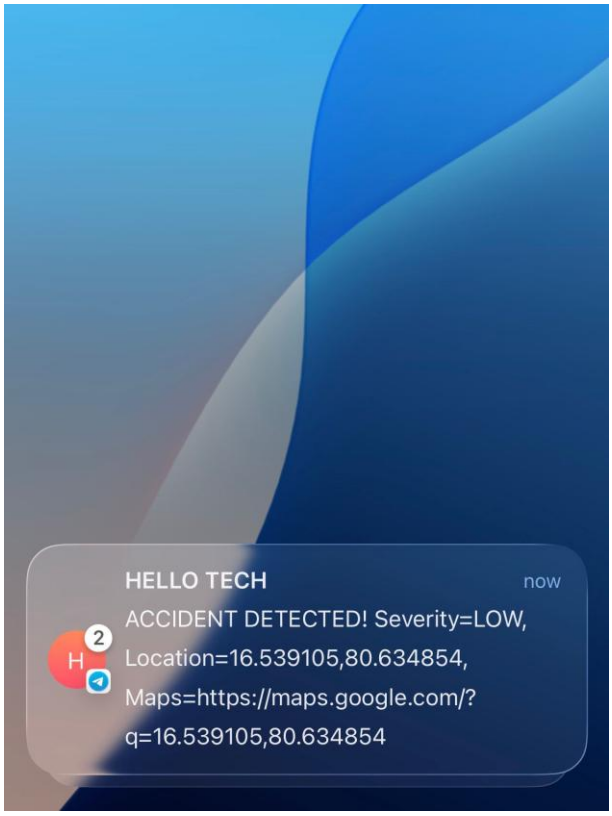


Figure 4.6: Alert Output Message-1 via text message

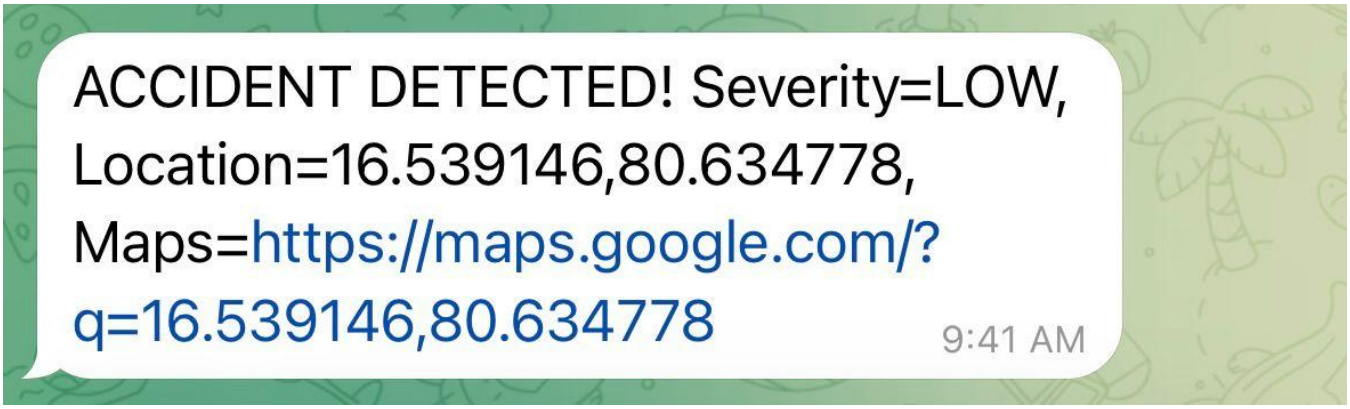


Figure 4.7: Alert Output Message-2 via Telegram

MODEL PERFORMANCE

The model was evaluated for three risk classes: LOW, MEDIUM, and HIGH. The controlled test set contains 180 samples with 60 samples per class. The reported results are: Accuracy 93.33%, Macro Precision 93.41%, Macro Recall 93.33%, and Macro F1-score 93.36%.

CONCLUSION/RECOMMENDATIONS

5. CONCLUSION

The proposed Real-Time Accident Monitoring and Risk Prediction System provides an effective solution for improving road safety by addressing the problem of delayed accident detection and emergency response. Traditional systems mainly detect accidents only after they occur, which may delay medical assistance and increase the severity of injuries. To overcome this limitation, the proposed system combines IoT-based sensing, weather-aware risk analysis, and LSTM-based sequence learning to support both real-time accident monitoring and risk prediction.

The system is built using an ESP32 microcontroller along with an MPU6050 accelerometer, GPS module, crash switch, panic button, LCD display, buzzer, and motor for vehicle representation. These components continuously monitor important parameters such as vehicle motion, sudden impact, crash switch status, panic input, and GPS location. The collected sensor data is then sent to a Python-based monitoring system for real-time processing and analysis.

To improve risk estimation, the Python-based system also considers live weather parameters such as rain, precipitation, wind speed, and visibility. A stacked LSTM model with a sliding window approach is used to analyze sequential sensor and environmental data and classify accident risk into LOW, MEDIUM, and HIGH levels. This helps the system identify dangerous conditions more effectively and adds intelligent prediction capability to the overall solution.

Whenever a risk or accident condition is detected, the system provides immediate local alerts through the LCD and buzzer, updates a live web dashboard, and sends Telegram emergency notifications with severity level and a Google Maps location link based on GPS coordinates. Overall, the project successfully demonstrates a practical hybrid approach that connects hardware-based real-time accident detection with software-based risk prediction and alert communication, helping to reduce response delay and improve road safety awareness.

FUTURE ENHANCEMENTS

Future Enhancements

The proposed **Real-Time Accident Monitoring and Risk Prediction System** can be further improved by integrating it with **official emergency response networks** such as **100/108** through authorized APIs. Instead of relying only on Telegram notifications, future versions can directly communicate with police, ambulance, or emergency service centers for faster and more reliable response. This enhancement would make the system more suitable for real-world deployment and improve the effectiveness of accident reporting during critical situations.

Another important future enhancement is the development of a **dedicated mobile application** for drivers, vehicle owners, and emergency contacts. A mobile app can provide **live monitoring of vehicle status, risk level, accident alerts, GPS location, and emergency notifications** directly on smartphones. It can also allow users to view past alerts, receive instant updates, and manually interact with the system when required, making the solution more user-friendly and accessible.

The system can also be improved by adding **cloud-based data logging and long-term accident-risk analysis**. In future versions, sensor data, alert history, and accident records can be stored in the cloud for continuous monitoring and analytics. This would help in identifying repeated risk patterns, analyzing accident-prone situations, and improving safety insights over time. In addition, collecting a **larger real-world dataset** from multiple vehicles can help improve the training of the **stacked LSTM model**, increasing prediction accuracy and making the risk classification more reliable in practical conditions.

Further enhancement can be achieved by integrating **camera-based accident context fusion** along with the existing sensor-based monitoring system. By combining visual information with motion, crash, GPS, and weather data, the system can provide richer accident context and more intelligent decision-making. The project can also be deployed in **real vehicles with improved enclosure design and power management**, making the hardware more durable, stable, and suitable for continuous field operation. These future enhancements can transform the proposed system into a more advanced, scalable, and real-world ready smart road safety solution.

REFERENCES

6.REFERENCES

[1] Y. Xu, H. Hu, C. Huang, Y. Nan, Y. Liu, K. Wang, Z. Liu, and S. Lian, "TAD: A Large-Scale Benchmark for Traffic Accidents Detection from Video Surveillance," IEEE Access, 2024.

[2] S. Hochreiter and J. Schmid Huber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. MIT Press, 2016.

[4] S. Chen, Y. Chen, and Y. Zhang, "Traffic accident detection based on convolutional neural networks," IEEE Transactions on Intelligent Transportation Systems, 2019.

[5] W. Yuan, J. Li, and H. Wang, "Vision-based traffic accident detection using deep learning," IEEE Access, 2018.

[6] A. Doshi and M. M. Trivedi, "On the roles of eye gaze and head pose in predicting driver's intent to change lanes," IEEE Transactions on Intelligent Transportation Systems, 2009.

[7] Y. Lv, Y. Duan, W. Kang, Z. Li, and F. Wang, "Traffic flow prediction with big data: A deep learning approach," IEEE Transactions on Intelligent Transportation Systems, 2015.

[8] Z. Zhao, W. Chen, X. Wu, P. C. Y. Chen, and J. Liu, "LSTM network: Adeep learning approach for short-term traffic forecast," IET Intelligent Transport Systems, 2017.

[9] H. Wang and X. Hu, "Deep learning for intelligent transportation systems: A survey," IEEE Transactions on Intelligent Transportation Systems, 2020.

[10] W. Luo, W. Liu, and S. Gao, "A revisit of sparse coding-based anomaly detection in stacked RNN framework," IEEE International Conference on Computer Vision, 2017.

[11] M. Alam, J. Ferreira, and J. Fonseca, "Introduction to intelligent transportation systems," Springer International Publishing, 2016.

[12] A. Edwards, "The relationship between road accident severity and recorded weather," Journal of Safety Research, 1998.

[13] World Health Organization, "Global status report on road safety," WHO, 2018.

[14] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," IEEE Conference on Computer Vision and Pattern Recognition, 2018.

[15] M. Abou-Zeid and M. Ben-Akiva, "Safety evaluation of driver behavior models," Transportation Research Part C, 2014.

[16] Y. Zhang, Y. Zheng, and D. Qi, "Deep spatio-temporal residual networks for citywide crowd flows prediction," Proceedings of the AAAI Conference on Artificial Intelligence, 2017.

[17] B. Yu, H. Yin, and Z. Zhu, “Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting,” Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI), 2018.

[18] Z. Yuan, X. Zhou, and T. Yang, “Hetero-ConvLSTM: A deep learning approach to traffic accident prediction on heterogeneous spatio-temporal data,” Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2018.

[19] L. Zhang, F. Chen, and J. Ma, “Deep learning for traffic prediction: Methods, applications, and challenges,” IEEE Transactions on Intelligent Transportation Systems, 2020.

[20] X. Ma, Z. Tao, Y. Wang, H. Yu, and Y. Wang, “Long short-term memory neural network for traffic speed prediction using remote microwave sensor data,” Transportation Research Part C: Emerging Technologies, 2015.

[21] A. Pande and M. Abdel-Aty, “Assessment of freeway traffic parameters leading to lane-change related collisions,” Accident Analysis and Prevention, 2006.

[22] M. Eisenberg, “The mixed effects of precipitation on traffic crashes,” Accident Analysis and Prevention, 2004.

PUBLISHED PAPER

8. PUBLISHED PAPER

TIJER || ISSN 2349-9249 || © April 2026 Volume 13, Issue 4 || www.tijer.org

Traffic Accident Risk Prediction Using LSTM with Vehicle Motion and Environmental Features

[1]Bejjupuramu Suseela, [2]Boggavarapu Manasa Lavanya, [3]Maddali Venkata Ragga Nandini, [4]Beesabathuni S N Manjunath, [5]Boyapati Venkata Suraj, [6]Dr.G.Krishna Kishore
[1]Student,[2]Student,[3]Student,[4]Student,[5]Student,[6]Assistant Professor
Department of Computer Science and Engineering
Dhanekula Institute of Engineering & Technology, Ganguru, A.P. (India) – 521139

Abstract—Road traffic accidents have emerged as a key threat to public safety causing heavy human and economic losses in all parts of the world. Because of the rapid urbanization and increase in road traffic, making the road safe became an important research problem. Intelligent transportation system has a potential for monitoring traffic conditions and reducing accident risk. Recent advances in deep learning allow the adoption of data driven approaches for traffic pattern analysis. Repeated use of LSTM and RNN models with sequence data and time-series data. It is suitable to use traffic sensor data where temporal correlation is essential for accident risk prediction.

This study suggested a deep learning framework for traffic accidents. A deep learning system has the capability of predicting risk of accident beforehand on the basis of an analyze of the sequential vehicle sensor data and environment conditions. We use a sliding window of 30 timestep for the sequence length to capture the temporal relationship of sensor observations. The input features include vehicle acceleration parameters, impact signals, precipitation, rainfall, wind speed and visibility. The motion behavior of the vehicle as well as ambient features causing an accident are well characterized by these features. Assisted by the stacked LSTM model with two LSTM layers with the help of concealed components and one dropout for preventing overfitting along with dense layers for multi-class risk classification the prediction model was implemented further into the study. The experiments revealed that the suggested system could learn a temporal traffic pattern and predict accident risk levels effectively. In the present intelligent transport scenario, deployment of such intelligent road safety systems can enhance decision-making and prevent accident pro- actively.

Index Terms— Traffic Accident Prediction, LSTM Networks, Deep Learning, Time Series Analysis, Intelligent Transportation Systems, Risk Classification, Sensor Data Analytics.

I. INTRODUCTION

Due to heavy use of vehicles, a lot of problems like road traffic crash, congestion, injury, fatality ,etc are raised [13]. For a long time, road safety hardly had any concern but with rapid raise of urbanization and vehicle usage, it did. Serious problems have arisen which have become a serious concern.

The rise of intelligent transport systems (ITS) presents a new era in traffic surveillance and accident detection [9], [11]. Intelligent Traffic Systems (ITS) closely monitor traffic through the use of live updates, artificial intelligence techniques, and state-of-the-art sensors. This contributes significantly to effective surveillance and accident detection, thus monitoring accident-prone zones continuously.

The emergence of traffic sensors, vehicle telemetry and environmental monitoring has led to the development of largescale traffic datasets, The datasets can provide valuable information about vehicle characteristics and road conditions, which can further be used to find the effect of environmental conditions on road accidents. With it, machine learning models can recognize patterns of risky behavior.

In recent times, successful applications of deep neural networks have been made in several transportation applications, such as traffic flow prediction, congestion detection, driver be haviour analysis and accident detection. Recent developments in sensor technology are resulting in increased availability of traffic data, but it is getting more complex.

The proposed work suggests a deep learning framework which integrates the vehicle motion data and the environmental features to predict accidents risk using LSTM. Using the sequential learning capabilities of LSTM networks, the proposed approach captures the temporal dependencies in sensor data to classify multiple risk levels of traffic conditions.

Conventional accident detection systems, whether scrutinized through human inspection or rule-based algorithms, detect abnormal incidents by recognizing a defined thresh old. Although easier compared to others, they can produce extremely ineffective results as they fail to seize complex patterns.

Lately, deep learning methods have been used to analyze traffic data. According to the source cited, it allows for higher accuracy in crash identification. The dataset called TAD was designed relying on the above work. This dataset can through deep learning-based algorithms [3], traffic data help create video analytics solutions for detecting vehicular accidents [1].

Most current techniques focus on detecting accidents after they've already occurred. Forecasting the traffic accident risks beforehand is much harder as it requires the sensing of temporal patterns and slight variations of vehicle behavior as well as environment condition. Researchers have studied LSTM models for traffic flow prediction, anomaly detection, and other applications. Long short-term memory networks are capable of learning long term dependencies. Long short-term memory techniques are appropriate for this task. This study presents the principal contribution of the study vehicle behavior.

- A deep learning system to predict traffic accident risks using sequential sensor data.
- Integration of environmental and vehicle motion features for improved prediction performance.
- Design of a stacked LSTM architecture capable of cap turning temporal traffic patterns.
- The parameters that determine the thresholds in the decision can also be learnt through the adaptive thresholding schemes.

II. RELATED WORK

The detection and forecasting of traffic accidents has garnered a lot of interest in computer vision [9], machine learning and intelligent transportation systems. In the beginning, typical statistical models and rule-based algorithms were used with preset thresholds to detect stray traffic events. The advancement of computer vision technologies, many researchers started using video surveillance data to detect accidents. These approaches mainly extract features and use machine learning [1], [5] classifiers to capture a collision event in a traffic scene. Researchers have been studying accident detection from traffic footage for long time. Traffic jams and damage to roads have become a huge problem for city planners. These models automatically learn visual features of abnormal traffic behavior[4]. The efficacy of traffic monitoring systems enhanced with deep learning. According to the vehicle insurance policy, damage costs for a severe accident will be fully covered if the other party pays up for total loss. Accidents involving automobile are related automobile insurance [1]. Usually, it is made up out of 4 wheels and in many states, 3-wheel. Along with video-based systems, sensor based accident detection systems have also gained attraction [6], [15].

These systems utilize information related to vehicle motion like its acceleration, orientation, impact signals, and many more to determine the. Multiple analyses of sequential data have been performed using an RNN and LSTM model [2]. Their ability to learn long-term dependencies makes them suitable for the analysis of traffic patterns and accident risk prediction. Even though these developments emerged, integrating environmental conditions with vehicle motion data is still a significant research challenge. Accident probability can be affected by weather such as rain, wind speed, poor visibility etc [12]. Recently, hybrid deep-learning models that integrate spatial and temporal learning processes have been implemented to predict road traffic [16]–[19]. What is more, existing studies have employed spatial mechanisms based on convolution and graph neural networks, whereas temporal mechanisms based on recurrent neural networks. Several research articles demon state that the combination of recurrent and convolutional networks can be advantageous in capturing the temporal dependency and spatial correlation present in traffic datasets. Recently, several studies have included existence studies of graphical neural. Numerous studies have examined how rainfall, fog, and low visibility contribute to road accidents and safety problems [12], [22]. . Current studies utilize either traffic movement signals or scenic with-object-traffic information in accident risk prediction. In contrast, our system proposes to combine visual and attention in a single deep network architecture [16]–[19]. This network gets the appearance and dynamic features from the driving scene.

III. METHODOLOGY

A. Input Features

Data acquisition stage is the first stage of the overall frame work of the proposed system. The stage when the readings of motion sensors and environmental sensors are taken is called the sensing stage. The sensors used to monitor the motion of the vehicle take in the vehicle acceleration along x and y axis along with collision impact sensors. Sensors also gather data about the amount of rainfall, wind speed and visibility. While preprocessing stage, the raw sensor data is cleaned and normalized. Missing values are interpolated, noise is filtered from the sensor signals and applying normalization to

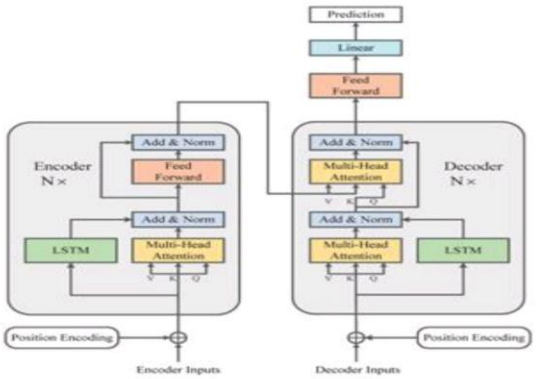


Fig. 1. Overall architecture of the proposed traffic risk prediction system.

features that enhances the classifier's accuracy. We preprocess all our features using Z score standardization, to ensure uniformity. The model performance gets improved with this step. Through the sliding window technique, fixed-length se sequences are generated from continuous time series data in this phase of the sequence generation. The vehicle count and speeds of the units to make a window. A set of historical observations on the sensor readings is contained in each window that contains the evolution of traffic over time. Each window is a sequence which will be passed as input. Ultimately, the deep learning stage classifies the level of risk and provides probabilities using a stacked LSTM network for the various accident risk categories. We have used nine features that are extracted from the vehicle's motion as input to the model. Sensors in the systems and environments. The vehicle's movement along three axes is reflected through the acceleration components (ax, ay, az), helpful in detecting sudden braking, turning and collision. Signaling impact refers to sudden force applied on the vehicle which may represent collision. Environmental features rain, precipitation, wind speed, and visibility conditions create circumstances and risk factors in crash statistics.

B. Data Preprocessing

Before training the model, the raw sensor data undergo several preprocessing steps.Data normalization is performed using Z-score standardization to ensure that all features have similar scales [3]. Sometimes sensor artifacts get embedded within the continuous data Using Noise Reduction, one can eliminate this noise and enhance model performance. Missing values are often dealt with interpolation techniques to make the data appear better.



Fig. 2. Workflow of the proposed traffic risk prediction system

C. Sequence Generation

The time series data have been converted to a sequence by using the sliding window technique. Every sequence has 30 timesteps of previous sensor readings. The model captures temporal changes in transportation patterns with the potential of leveraging LSTM's temporal pattern capturing mechanism.

D. LSTM Model Architecture

The deep learning architecture incorporates stacked LSTM layers featuring 64 hidden units for effective training and modeling. A layer learns higher level temporal dependencies while the other layer directly learns temporal features from input sequences. To reduce, a dropout layer drop-out 0.2 is utilized. Excessive fitting and model generalization. The end result of the. The LSTM layers pass through a 32-neuron dense layer. ReLU activator. Subsequently, a softmax layer identifies the traffic situation as Low Risk, Medium Risk or High Risk.

V. EXPERIMENTS AND RESULTS

The model we developed from the most recent dataset input was tested and designed on continuous sensor readings and other environmental variables. The dataset utilized training ratios of 70:15:15 for training, validation, and testing.

The performance of the proposed model is assessed using several performance measures such as accuracy, precision, recall, and F-measure. The correctness of the model overall on the provided dataset is known as accuracy, whereas the correctness of the positive predictions is known as precision. Recall demonstrates the capacity of a model to retrieve all pertinent instances. Likewise, the F1-score shows the balance between precision and recall.

The training of a model should be monitored to prevent overfitting, which helps in creating a better model that generalizes well. Monitoring the training process is best done with validation data. The curves for training and validation accuracy are approaching identical values.

The result indicates that the model learns time series patterns from sequential sensor data. The main point of your essay is good, but you must use paragraphs. A paragraph should contain one main point and discuss that point fully. My initial paragraph, for example, would be dedicated to how it affects our society. My second life-changing event essay would be about what happened.

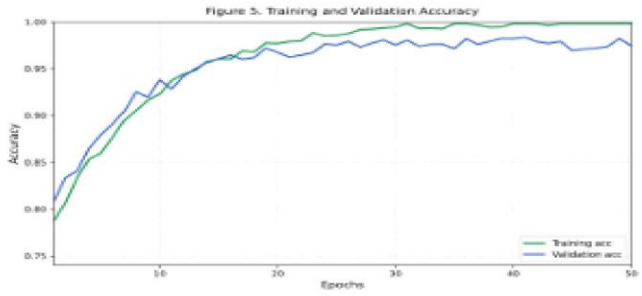


Fig. 5. Training and validation accuracy of the LSTM model.

The intelligent system development that processes data from sensors for elder care has been the subject of considerable interest recently. The activities of elders and their environment.

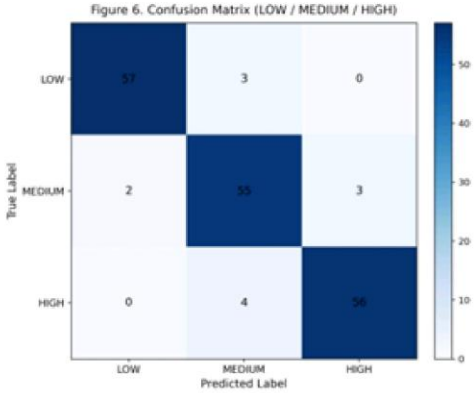


Fig. 6. Confusion matrix showing classification performance for three risk levels.

The confusion matrix of the analysis above shows that the model has high classification accuracy for all 3 risk classes.

VI. DISCUSSION

The experimental evaluation results confirm the effective ness of the proposed framework on traffic risk prediction with LSTMs. The model captures complex temporal patterns associated with accident risks by analyzing sequential sensor data and features from the surrounding environment.

The proposed method allows us to combine various data sources, an important advantage of the proposed method. By combining the vehicle motion signal and environmental condition, it is able to capture more comprehensive and representative input. The sliding window technique helps the model in figuring out short-term temporal patterns. It also helps in identifying sudden changes in the driving condition. These unforeseen changes may cause an accident. Stacked LSTM layers increase the chance that the model captures hierarchical temporal features. The framework we propose also has a significant advantage of working with online, real time sensor input. With the rise of more connected vehicles on the road and smarter transportation infrastructure, the prediction system may be incorporated in real time monitoring systems. As a result, traffic management people can timely recognize the threat and take precautions accordingly. An opportunity of extension exists for the present explains approach. The extra sources of data such as traffic camera video, GPS trajectory and V2V communication signal can be fused with forensic training data to improve prediction quality.

VII. CONCLUSION

Every trip, whether a leisurely stroll or a ride, always comes with a bit of risk. Danger no longer limits it to human beings; it also puts life in threat. regarding animals Moreover, different travels such as walking In order to capture traffic temporal aspect windowing sliding of 30 time periods stacked LSTM model is used. The experiments proved that the traffic system can classify the traffic condition as low, medium and high risk. Rainfall, precipitation, the wind speed, and visibility allow the model to evaluate accident risk in various road conditions. Data sources may be added to our proposed framework in future work, e.g., videos taken from traffic cameras, GPS trajectories, and V2I signals. In the future, we expect that including these heterogeneous data types will enhance our systems performance over more challenging traffic scenarios. We will explore the application of cutting-edge deep learning model classes like attention based networks, GNNs, CNN-LSTMs for superior predictive performance in future experiments. The objective is to implement intelligent traffic monitoring systems which can predict accidents in real-time to make transport safer.

REFERENCES

[1] Y. Xu, H. Hu, C. Huang, Y. Nan, Y. Liu, K. Wang, Z. Liu, and S. Lian, “TAD: A Large-Scale Benchmark for Traffic Accidents Detection From Video Surveillance,” IEEE Access, 2024.

[2] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. MIT Press, 2016.

[4] S. Chen, Y. Chen, and Y. Zhang, “Traffic accident detection based on convolutional neural networks,” IEEE Transactions on Intelligent Transportation Systems, 2019.

[5] W. Yuan, J. Li, and H. Wang, “Vision-based traffic accident detection using deep learning,” IEEE Access, 2018.

[6] A. Doshi and M. M. Trivedi, “On the roles of eye gaze and head pose in predicting driver’s intent to change lanes,” IEEE Transactions on Intelligent Transportation Systems, 2009.

[7] Y. Lv, Y. Duan, W. Kang, Z. Li, and F. Wang, “Traffic flow predic tion with big data: A deep learning approach,” IEEE Transactions on Intelligent Transportation Systems, 2015.

[8] Z. Zhao, W. Chen, X. Wu, P. C. Y. Chen, and J. Liu, "LSTM network: Adeep learning approach for short-term traffic forecast," IET Intelligent Transport Systems, 2017.

[9] H. Wang and X. Hu, "Deep learning for intelligent transportation systems: A survey," IEEE Transactions on Intelligent Transportation Systems, 2020.

[10] W. Luo, W. Liu, and S. Gao, "A revisit of sparse coding based anomaly detection in stacked RNN framework," IEEE International Conference on Computer Vision, 2017.

[11] M. Alam, J. Ferreira, and J. Fonseca, "Introduction to intelligent transportation systems," Springer International Publishing, 2016.
[12] A. Edwards, "The relationship between road accident severity and recorded weather," Journal of Safety Research, 1998.

[13] World Health Organization, "Global status report on road safety," WHO, 2018.

[14] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," IEEE Conference on Computer Vision and Pattern Recognition, 2018.

[15] M. Abou-Zeid and M. Ben-Akiva, "Safety evaluation of driver behavior models," Transportation Research Part C, 2014.

[16] Y. Zhang, Y. Zheng, and D. Qi, "Deep spatio-temporal residual net works for citywide crowd flows prediction," Proceedings of the AAAI Conference on Artificial Intelligence, 2017.

[17] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional net works: A deep learning framework for traffic forecasting," Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI), 2018.

[18] Z. Yuan, X. Zhou, and T. Yang, "Hetero-ConvLSTM: A deep learning approach to traffic accident prediction on heterogeneous spatio-temporal data," Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2018.

[19] L. Zhang, F. Chen, and J. Ma, "Deep learning for traffic prediction: Methods, applications, and challenges," IEEE Transactions on Intelligent Transportation Systems, 2020.

[20] X. Ma, Z. Tao, Y. Wang, H. Yu, and Y. Wang, "Long short-term memory neural network for traffic speed prediction using remote microwave sensor data," Transportation Research Part C: Emerging Technologies, 2015.

[21] A. Pande and M. Abdel-Aty, "Assessment of freeway traffic parameters leading to lane-change related collisions," Accident Analysis and Prevention, 2006.

[22] M. Eisenberg, "The mixed effects of precipitation on traffic crashes," Accident Analysis and Prevention, 2004